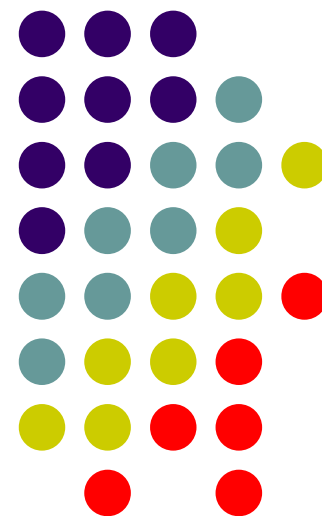
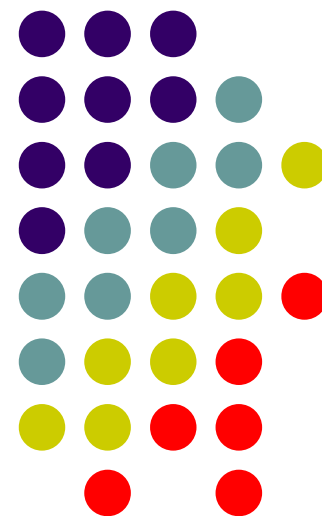
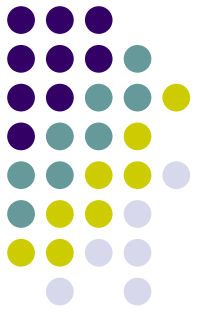


Ch2 Linux 命令基础



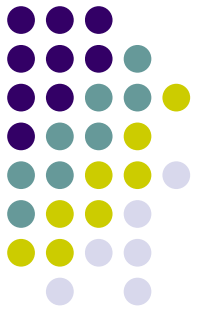
Ch2 Linux 命令基础





本章内容

- Linux 命令行与shell
- 用户管理
- 文件管理
- 进程管理
- 网络管理
- 软件包管理
- 文本编辑器vim



Part 1 Linux命令行与 shell

- Linux shell
- Linux命令格式
- Linux基础命令
- Linux管理命令



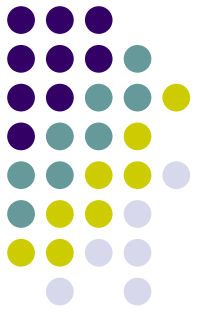
1.1 Linux Shell

- Shell就是系统的用户界面，提供了用户与内核进行交互操作的一种接口。
- Linux下可用的Shell也有很多种，如Bourne Shell、C Shell、Korn Shell、POSIX Shell以及**Bourne Again Shell (Bash)** 等



1.1 Linux Shell

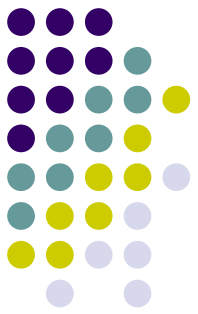
- Linux的bash相当智能化，可使用<TAB>键的自动补齐功能，将部分命令补充完整。
- bash还具备完善的历史记录功能。上下箭头
- 除了智能化外，bash的自定义功能也很强。



1.1 Linux Shell

- **Linux bash常用通配符**

- *: 匹配任意一个或多个字符
- ? : 匹配任意一个字符
- []: 匹配包含在[]内的单个字符



1.1 Linux Shell

- **Linux Bash常用通配符**

例: `ls *.txt`

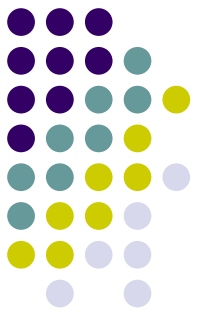
`ls ?.txt`

`ls ???.txt`

`ls /dev/sda[1-5]`

`ls [0-9]?.conf`

`ls [xyz]*.txt`

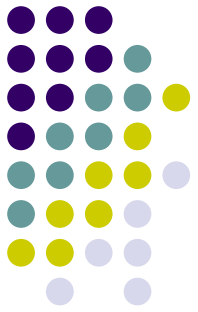


1.2 Linux命令格式

- **Linux命令格式**

command [options] [arguments]

命令 选项 参数



1.3 Linux基础命令

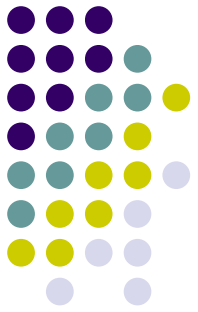
- **查看系统版本号**

1、查看操作系统发行版本号：

```
cat /etc/redhat-release
```

2、查看操作系统内核发行号：

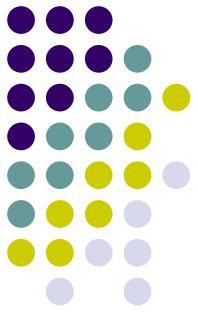
```
uname -r
```



1.3 Linux基础命令

- **查看系统配置**

- 1、查看cpu: `lscpu`
- 2、查看内存: `free`
- 3、查看磁盘: `fdisk -l`
- 4、查看网卡: `ip addr`



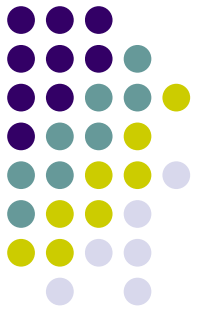
1.3 Linux基础命令

- 查看系统时间

1.date命令

命令格式: date

说明: 显示及设置日期和时间



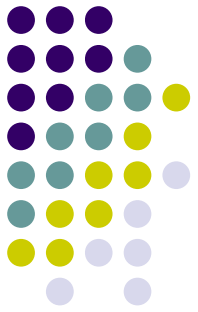
1.3 Linux基础命令

- 查看系统时间

2.cal 命令

命令格式: cal

说明: 显示某个月份的日历



1.3 Linux基础命令

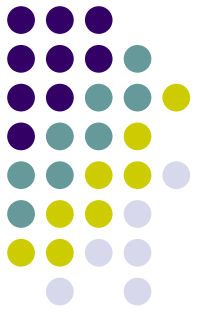
- 字符显示命令

1.date命令

echo 命令

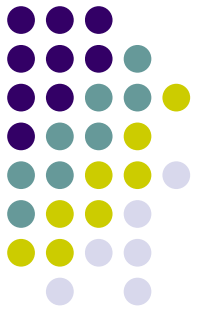
命令格式：echo [选项] [输出内容]

说明：显示字符串



1.4 Linux管理命令

- 文件管理
- 用户管理
- 进程管理
- 系统管理
- 网络管理
- 磁盘管理



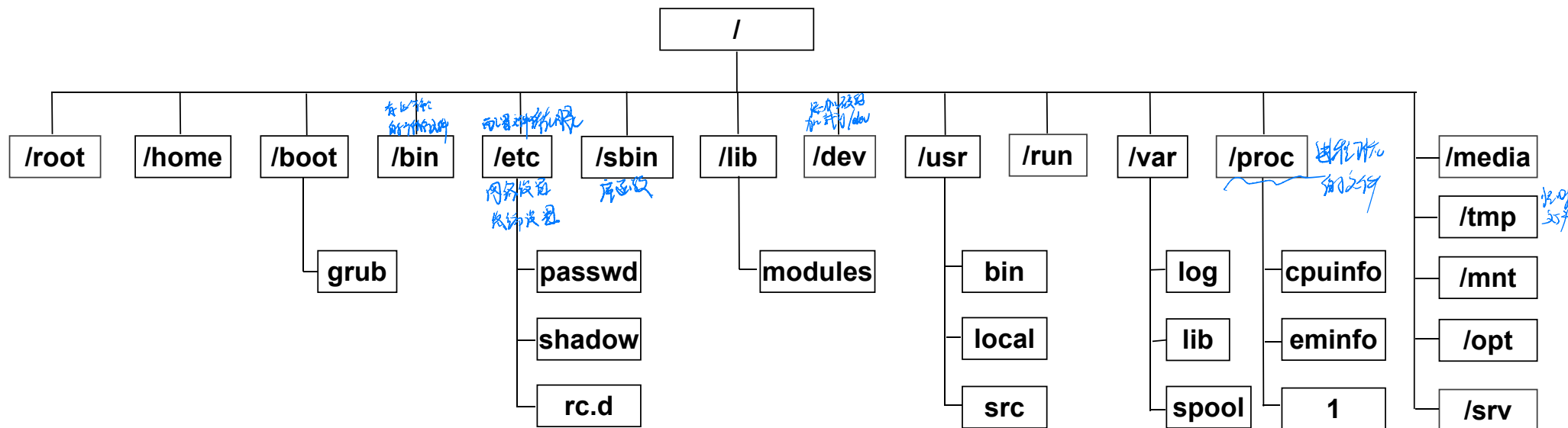
Part 2 文件管理

- 文件系统
- 文件和目录
- 文件内容查看
- 文件搜索
- 文件权限
- 文件压缩解压

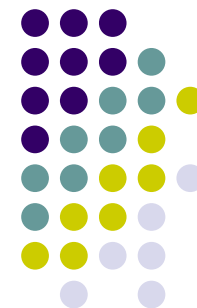
2.0 文件系统

Linux文件系统

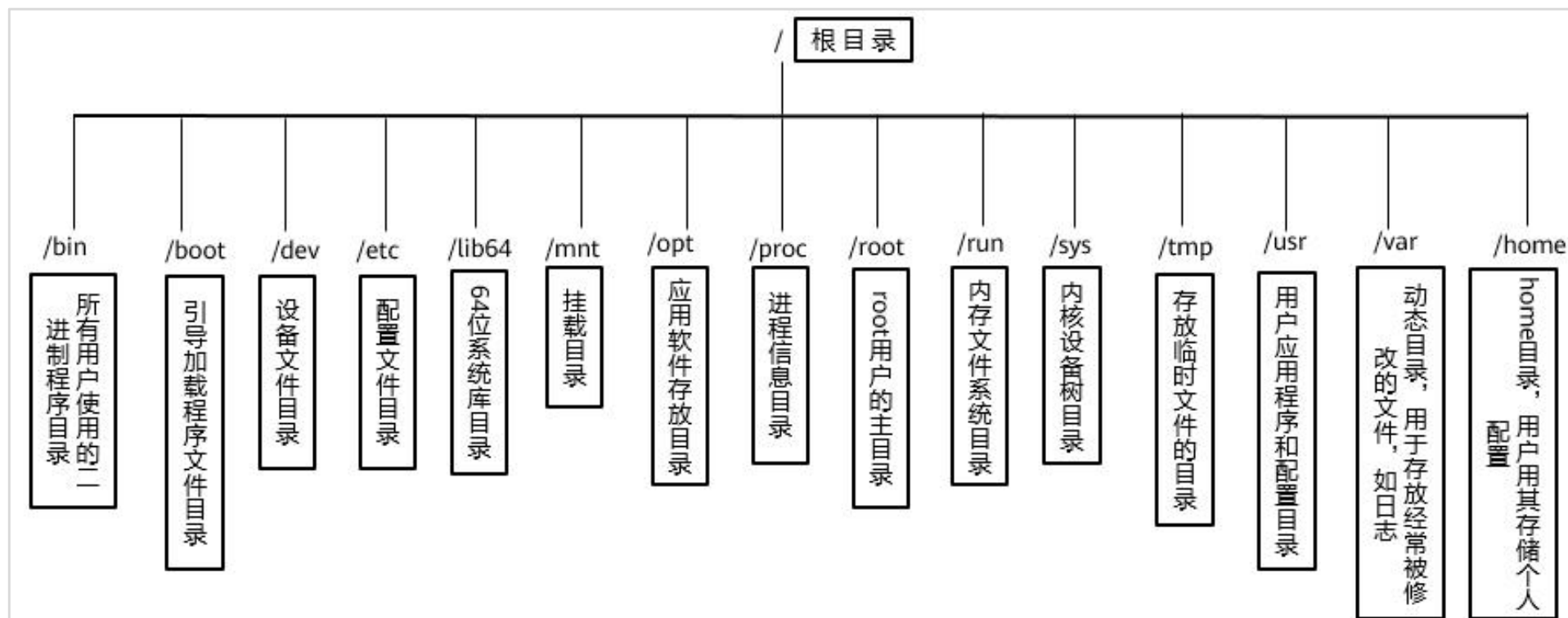
- Linux的文件系统采用**分层结构**。
- 其顶层为**根目录**，用符号“/”表示，在根目录下是不同的子目录。

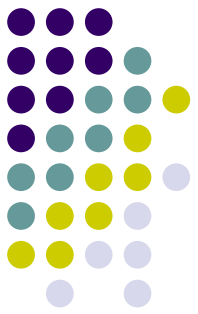


2.0 文件系统



● Linux文件系统





2.0 文件系统

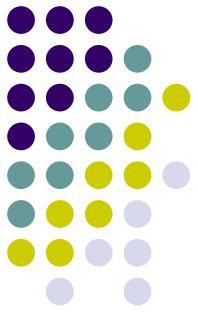
- 文件类型

文件类型分7类：

- 使用 “ls -l” 显示文件列表时，第1个字符则代表文件的类型



```
mylinux@ubuntu:~$ ls -l
total 56
drwxr-xr-x 2 mylinux mylinux 4096 Jul 29 18:11 Desktop
drwxr-xr-x 2 mylinux mylinux 4096 Jul 29 18:11 Documents
drwxr-xr-x 2 mylinux mylinux 4096 Jul 29 18:11 Downloads
-rw-rw-r-- 1 mylinux mylinux 0 Aug 1 00:34 file
-rw-rw-r-- 1 mylinux mylinux 6 Oct 28 20:08 file1
-rw-rw-r-- 1 mylinux mylinux 0 Oct 28 20:03 file2
-rw-rw-r-- 1 mylinux mylinux 12 Oct 28 18:51 file3
-rw-rw-r-- 1 mylinux mylinux 6 Oct 28 20:03 file4
```



2.0 文件系统

● 文件类型

1. 普通文件 (-)
2. 目录文件 (**directory**)
3. 块设备文件 (**block**)
4. 字符设备文件 (**character**) 键盘、串口.
5. 符号链接文件(**link**)
6. 命名管道文件 (**pipe**) 有名管道.
7. 套接字文件 (**socket**)

文件类型	符号	文件类型	符号
普通文件	-	符号链接文件	link
目录文件	directory	命名管道文件	pipe
块设备文件	block	套接字文件	socket
字符设备文件	character		



2.0 文件系统

● 文件名

- 由字母（可用汉字）、数字、下划线、圆点等字符构成，除字符 “/” 外，所有的字符都可以使用。
- 长度不超过255个字符。
- 同一目录下不能有相同的文件名，不同目录下可以同名。
- 若文件名的第1个字符为 “.”，表示该文件为**隐藏文件**。 *ls -a*
- 文件的属性与取名无关，文件名中可以不规定扩展名。
- Linux系统中目录名、文件名是区分大小写的。

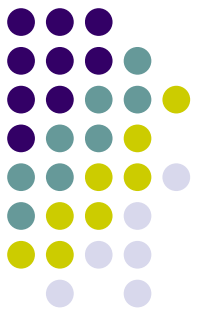


2.0 文件系统

- **文件目录**

- 目录：指包含许多文件项目的一类特殊文件。子目录、父目录、**工作目录**、**用户主目录(Home Directory)**。
- “.” 表示当前目录， “..” 表示父目录， “~” 表示用户主目录。

家目录



2.0 文件系统

- **文件路径**

- **路径**：由目录名和 “/” (斜杠)做分隔符组成的字符串

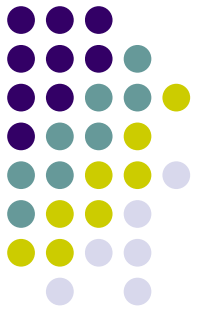
- **绝对路径**

有的都习惯用绝对路径。

由根目录 / 开始，例如：/usr/share/doc, /root和
/home/mylinux

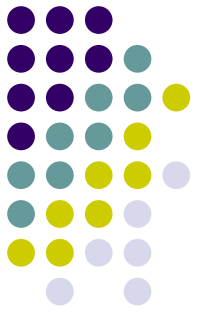
- **相对路径**

不是由根目录 / 写起，而是由当前目录开始，例
如：../java



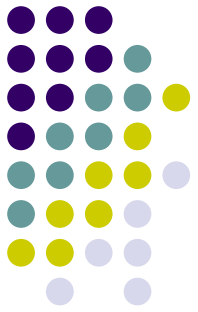
2.1 文件和目录操作命令

- pwd
- cd
- ls
- touch
- mkdir
- cp
- rm
- mv
- mkdir



2.1.1 pwd

- 命令原意: print working directory
- 命令功能: 打印当前工作目录的绝对路径
- 命令格式: pwd



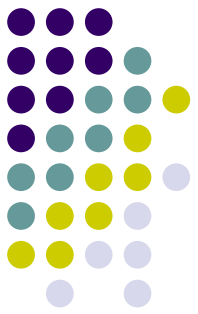
2.1.2 cd

- 命令原意: change directory
- 命令功能: 切换目录
- 命令格式: cd 参数

cd 切换目录
cd - 回到上次目录

参数	说明
.	切换到当前目录
..	切换到父目录
~	切换到用户家目录

/home/ly

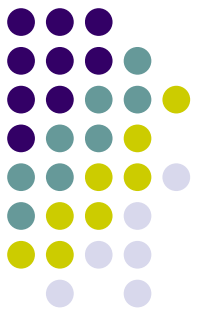


2.1.3 ls

- 命令原意: list
- 命令功能: 列出参数的属性信息
- 命令格式: ls [选项] [参数]

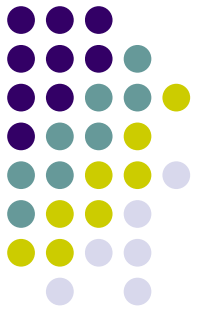
参数	说明
-l	显示当前目录的详细信息
-a	显示当前目录下所有文件包括隐藏文件
-i	显示文件的inode编号
-R	递归显示当前目录下包括子目录下的所有文件信息

ls -d 显示当前目录本身的信息



2.1.4 touch

- 命令功能：更新已存在文件的时间标签，若文件不存在则新建文件
更新为当前时间
- 命令格式：touch 参数
- 示例：touch hello.c

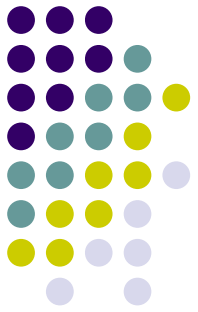


2.1.5 mkdir

- 命令原意: make directory
- 命令功能: 创建目录
- 命令格式: mkdir [选项] 参数

参数	说明
-p	递归创建目录

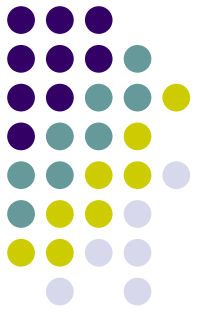
1 a / b / dir / ...
→ 若不存在路径, 可创建好



2.1.6 cp

- 命令原意: copy
- 命令功能: 将一个或多个源文件复制到指定目录
- 命令格式: cp [选项] 源文件或目录 目标目录

参数	说明
-R	递归处理，包括子目录下的文件
-f	强制复制文件或目录

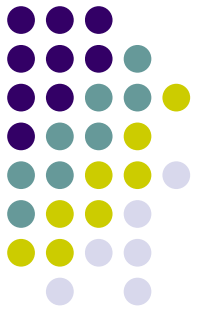


2.1.7 rm

- 命令原意: remove
- 命令功能: 删除目录中的文件或目录
- 命令格式: rm [选项] 文件或目录

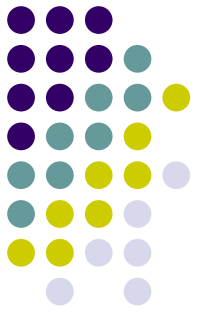
参数	说明
-f	强制删除文件或目录 <i>force</i>
-i	删除文件前进行询问
-rf	强制删除所有文件和子目录, 不会询问

-rf



2.1.8 mv

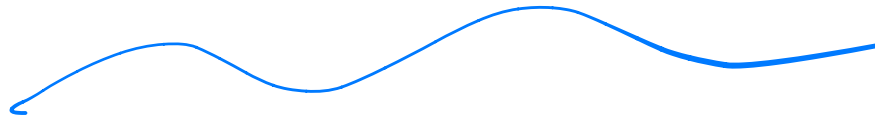
- 命令原意: move
 - 命令功能: 移动文件或目录
 - 命令格式: mv 源文件或目录 目标目录
- 



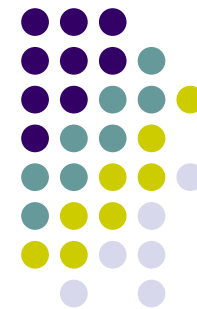
2.1.9 rmdir

- 命令原意: remove directory
- 命令功能: 删除目录
- 命令格式: rmdir [-p] 目录

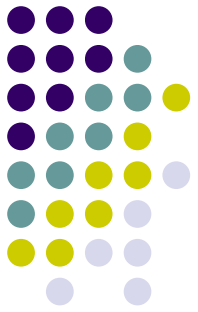
只能删除空目录。



2.2 文件内容查看命令



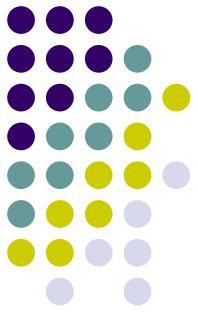
- cat
- more
- less
- head
- tail
- wc



2.2.1 cat

查看文件内容

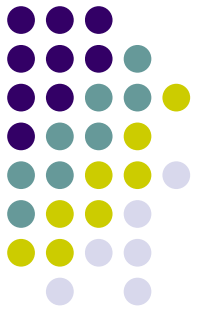
- 命令原意: concatenate and display files
- 命令功能: 打印文件内容到输出设备
- 命令格式: cat 文件名



2.2.2 more

- 命令功能：分页显示文件内容
- 命令格式：more 文件名

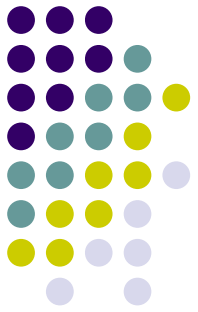
快捷键	用途
Enter	向下n行，需要定义。默认为1行。
Ctrl+F	向下滚动一屏
空格键	向下滚动一屏
Ctrl+B	向上滚动一屏
b	向上滚动一屏
=	输出当前行号
:f	输出文件名和当前行号
q	退出more



2.2.3 less

- 命令功能：分页显示文件内容，可实现向上翻页
- 命令格式：less 文件名

快捷键	用途
b	向上翻一页
d	向下翻半页
h	显示帮助界面
q	退出less
u	向上翻半页
y	向上翻一行
空格键	向下翻一行
Enter	向下翻一页
上下键	向上/下翻一行



2.2.4 head

不加-n的话.
默认查看前10行

- 命令功能：查看文件的前n行
- 命令格式：head -n 文件名

✓ 显示/etc/passwd文件的前20行。

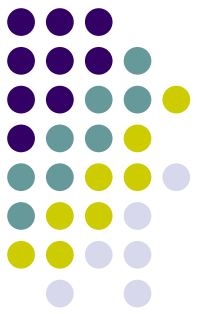
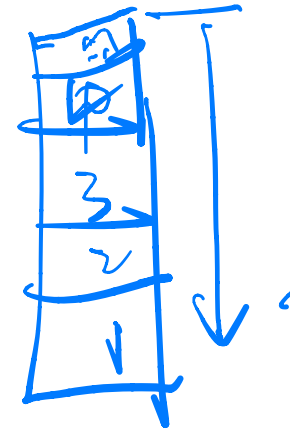
```
[root@host ~]# head -n 20 /etc/passwd
```

✓ 显示/etc/passwd文件的除了最后20行的前面所有行，此时数值变量可以用负值表示，若该文件一共是37行，则会显示前17行内容。

```
[root@host ~]# head -n -20 /etc/passwd
```

2.2.5 tail

- 命令功能：查看文件的后n行
- 命令格式：tail -n 文件名



- ✓ 显示/etc/passwd文件的后20行。

```
[root@host ~]# tail -n 20 /etc/passwd
```

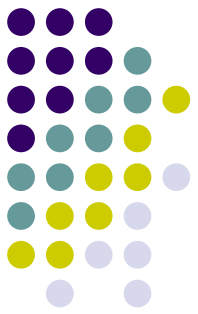
- ✓ 显示/etc/passwd文件的2行开始之后的所有行，此时在变量数值前加入“+”号表示，若该文件一共是37行，则会显示2行到37行所有内容。

```
[root@host ~]# tail -n +2 /etc/passwd
```

- ✓ 显示/etc/passwd文件的第11到第20行。

```
[root@host ~]# head -n 20 /etc/passwd | tail -n 10
```

前10



2.2.6 wc

- 命令原意: word count
- 命令功能: 计算文件的字节数、字数和列数
- 命令格式: ~~wc [选项]~~ 参数

示例:

wc -l /etc/passwd

#统计passwd文件的行数

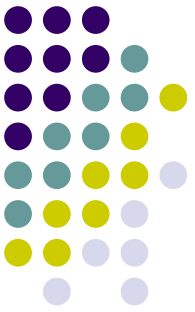
wc -c /etc/passwd

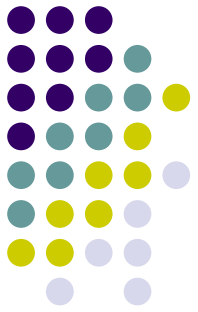
#统计passwd文件的字节数

参数	说明
-c <i>character</i>	统计指定文件中文本的字节数
-l <i>line</i>	统计指定文件中文本的行数
-w <i>word</i>	统计指定文件中的字数

2.3 文件搜索命令

- which
- find
- locate
- grep



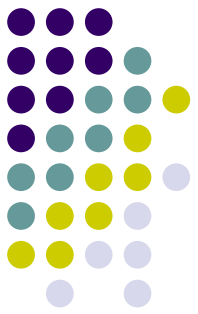


2.3.1 which

- 命令功能：查看命令所在路径
- 命令格式：which 命令

示例：which ls

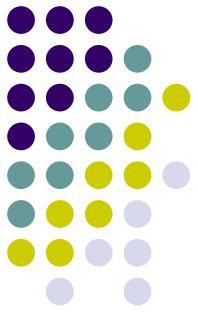
ls



2.3.1 find

- 命令功能：借助搜索关键字（文件名、文件大小、文件所有者等）查找文件或目录
- 命令格式：find 搜索路径 [选项] 搜索关键字
- 示例：find /etc -name passwd

选项	说明
-amin	按文件存取时间来查找文件，-n指n分钟以内，+n指n分钟以前
-atime	按文件存取时间来查找文件，-n指n天以内，+n指n天以前
-cmin	根据文件修改（文件属性和文件内容）时间来查找文件
-ctime	根据文件修改（文件属性和文件内容）时间来查找文件
-mtime	根据文件内容修改时间来查找文件
-type	查是块设备、目录、字符设备、管道、符号链接、普通文件



2.3.1 find

-exec 命令 {} \;

选项	说明
-exec	对查找的文件执行Linux命令，不询问用户 格式：-exec 要执行的命令 {} \;
-ok	对查找的文件执行Linux命令，询问用户是否执行
-follow	不包括符号链接
-uid	查找指定的用户id的文件
-user	根据文件所有者查找
-name	根据文件名查找
-size	根据文件大小查找，+n, n, -n
-perm	根据文件权限查找
-a	前后条件都要满足
-o	满足一个条件



2.3.1 find

- find使用举例

1.按文件名-name查找

```
#在/tmp 目录下查找下名字为“sample”的文件
$find /tmp -name "sample" -print
#在/tmp 目录下查找下名字以“.txt”结尾的文件
$find /tmp -name "*.txt" -print
#在当前工作目录下查找名字以大写字母开头的文件
$find -name "[A-Z]*" -print
#在/etc 目录下查找名字长6个字符 并以“.bak”结尾的文件
$find /etc -name "???.bak" -print
#在当前用户家目录下查找名字以file开头（忽略大小写）的目录文件
$ find ~ \( -type d -a -iname "file*.txt" \)
```

取C 7个字符



2.3.1 find

- find使用举例

2.按用户-user查找

#在/home 下查找属于用户 user01 的文件

find /home -user user01

#在/home 下查找文件拥有者 UID 大于 1001 的文件

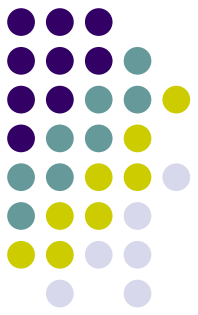
#find /home -uid +1001

#在/var 下查找从属用户组为 apache 的文件

#find /var -group apache

#在/home 下查找从属用户组 GID 为 1000，且文件名以小写字母或者数字开头的文件

#find /home \(-gid 1000 -a -name "[0-9a-z]*" \)



2.3.1 find

- find使用举例

3.按文件类型-type查找

#在当前目录下查找所有子目录

```
$find . -type d
```

#在当前目录下查找所有非目录文件

```
$find . !-type d -print
```

#在/dev 目录下查找所有的块设备和字符设备文件

```
#find /dev /( -type b -o -type c \)
```

通非的个

非! -d type



2.3.1 find

- find使用举例

4.按文件大小-size查找

#在当前工作目录下查找大小大于 100 字节的文件

\$find -size +100c

#在当前工作目录下查找大小小于 1k 字节的文件

\$find -size -1k -print

#在根目录下查找大于 900M 字节，小于 1G 字节的文件

#find / -size +900M -a -size -1G

#在当前工作目录下查找大小超过 10 块的文件 (1 块=512 字节)

\$find -size +10 -print

#在当前用户家目录下查找空文件或空目录

\$find ~ - empty



2.3.1 find

- find使用举例

5.按时间查找

#在当前目录下查找在 10 分钟之前, 15 分钟内访问过的文件

\$find /tmp -amin +10 -amin -15

#在当前目录下查找在 1 天前, 2 天内访问过的文件

\$find -atime +1 -atime -2

#在当前目录下查找在 10 分钟之前, 15 分钟内文件属性/内容发生改变的文件

\$find -cmin +10 -cmin -15

#在当前目录下查找在 1 天前, 2 天内文件属性/内容发生改变的文件

\$find -ctime +1 -ctime -2

#在当前目录下查找在 10 分钟之前, 15 分钟文件内容发生改变的文件

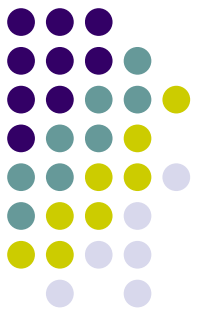
\$find -mmin +10 -mmin -15

#在当前目录下查找在 1 天前, 2 天内文件内容发生改变的文件

\$find -mtime +1 -mtime -2

#在用户家目录下查找文件内容更新时间比 tmp.txt 近的文件或目录

\$find /home -newer tmp.txt



2.3.1 find

- 在 Linux 中与文件相关的时间属性有三个：
- ✓ atime: 文件最后被访问时间, atime 是在读取文件或者执行文件时更改的。
- ✓ mtime: 文件内容最后被更改的时间, mtime 是在写入文件时随内容的更改而更改的
- ✓ ctime: 文件状态 (属性或者内容) 最后被更改的时间是在写入文件、更改所有者、权限或链接设置时随 inode 的内容更改而更改的。



2.3.1 find

- find使用举例

6.查找后执行命令

#在当前目录下找到名称为 tools 的对象,将这些文件（包括完整路径的）全名输出（默认动作）

```
$find -name tools -print
```

#在当前目录下找出大于 3KB 小于 3MB 的文件,并用 ls -dils 格式列出

```
$find -size +3k -size -3M -ls
```

#在当前目录下查找大小为 0 字节的文件,并把这些文件拷贝到 test 目录下

```
$find ./ -type f -size 0c -exec cp {} ./test/ \;
```

#在当前目录下查找大小为 0 字节的文件,按照 ls -lt 命令格式列出,并输出到文件 result.txt 文件中

```
$find ./ -type f -size 0c -fprint ./result.txt -exec ls -lt {} \;
```

#在/tmp 下查找名字以.txt 结尾的文件,并将这些文件打包为 text.tar 文件

```
$find /tmp -name *.txt -exec tar -uzf text.tar {} \;
```

#在/tmp 下查找名字以.tmp 结尾的文件,并删除之

```
$find /tmp -name tmp.txt -ok rm {} \;
```



2.3.1 find

- find使用举例

6.逻辑操作

与 -a

或 -o

非 !

#在用户家目录下查找文件名以为.tmp 或 .bak 结尾的文件

```
$ find ~ -name *.tmp -o -name xtmp*
```

#在/tmp 目录下查找名字以数字开头, 且类型非目录的文件

```
$ find /tmp \( -name [0-9]* -a ! -type d \)
```

#在当前工作目录下查找属于用户 stu 的,且类型为字符设备,名字以.tmp 结尾的文件

```
$ find \( -name *.tmp -a -type c -a -user stu \)
```




2.3.1 find

- 在 find 命令中可以使用如下逻辑运算符:

- ✓ -a 表示与
- ✓ -o 表示或
- ✓ ! 表示取反

使用多个条件时,这些条件要用圆括号括起来,左右圆括号还要使用反斜杠"\"来进行转义,以防止 shell 的解释;



2.3.3 locate

- 命令功能：借助搜索关键字查找文件或目录
- 命令格式：locate [选项] 搜索关键字
- locate与find的区别

locate速度远胜find

find搜索整个目录，locate搜索数据库/var/lib/locatedb

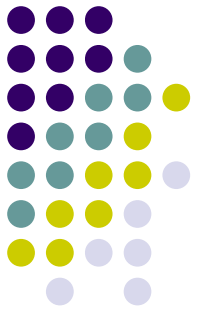
即便文件存在，数据库中没有记录，locate便搜索不到

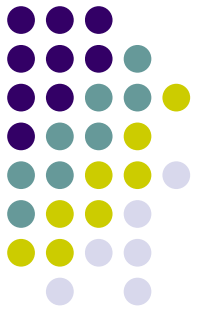
数据库文件默认一天更新一次，可用updatedb命令更新

2.3.4 grep

按行

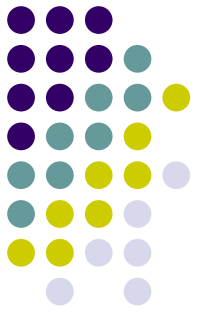
- 命令功能：在文件中搜索与字符串匹配的行并输出
- 命令格式：grep 指定字符 源文件
- 示例：grep user /etc/passwd





2.4 文件权限

- chmod *m*
- chown *设置/ owner*
- chgrp *group*



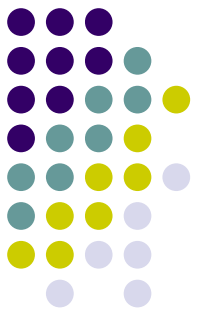
2.4 权限管理

- 文件的访问权限

读(r): 允许读文件的内容

写(w): 允许向文件中写入数据

执行(x): 允许将文件作为程序执行

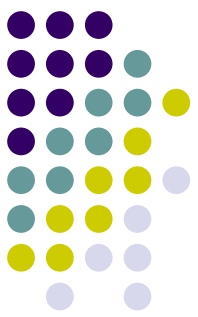


2.4 权限管理

- 根据用户与文件的关系，Linux系统中将用户分为文件或目录的拥有者、同组用户、其他组用户和全部用户；又根据用户对文件的权限，将用户权限分为读取权限（read）、写入权限（write）和执行权限（execute）。

权限	对文件的影响	对目录的影响
r (读取)	可读取文件内容 <i>read - cat</i>	可列出目录内容
w (写入)	可修改文件内容 <i>write vim</i>	可在目录中创建删除内容
x (执行)	可作为命令执行	可访问目录内容
目录必须拥有x权限，否则无法查看其内容		

即/创建组
与同组
与不同组

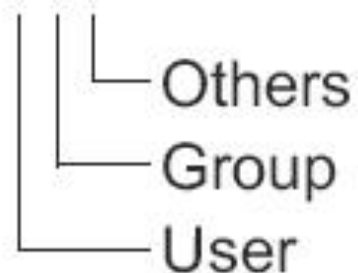


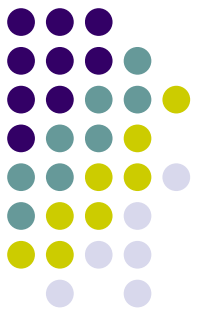
2.4 权限管理

- File and directory permissions can also be specified as an octal number:

	User	Group	Other
Symbolic notation	rwX	rw-	r-X
Binary	111	110	101
	4+2+1	4+2+0	4+0+1
Octal	7	6	5

```
$ chmod 765 file
```





2.4 权限管理

- 使用权限管理命令要求用户具有执行相应命令的权限，为保证命令可成功执行，先使用su命令，将用户切换为root。



2.4 权限管理

- 访问权限举例
- 要求用数字表示法表示右图中所列文件和文件夹的权限。

① ② ③

```
mylinux@ubuntu:~$ ll /etc/passwd /etc/shadow
-rw-r--r-- 1 root root 2856 Jul 29 18:18 /etc/passwd
-rw-r----- 1 root shadow 1417 Jul 29 18:18 /etc/shadow
mylinux@ubuntu:~$ ll -d /home /boot
drwxr-xr-x 4 root root 4096 Oct 28 17:52 /boot/
drwxr-xr-x 3 root root 4096 Jul 29 17:33 /home/
```

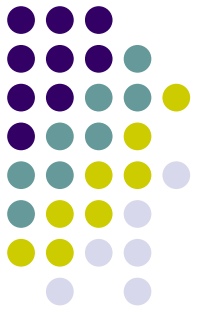
844 840 755 755

111 101 101

755

2.4.1 chmod

修改权限



- 命令原意: change the permissions mode of file
- 命令功能: 变更文件或目录的权限
- 命令格式: chmod {augo}{+ -=} 文件或目录
- 示例:

chmod u-x, g-x file

chmod 754 file

• The change mode command:

```
$ chmod <who operator what> filename
```

who:

u = owner of file

g = group

o = other users on the system

a = all (owner+group+others)

operator:

+ = add permission

- = remove permission

= = clear permissions and set to mode specified

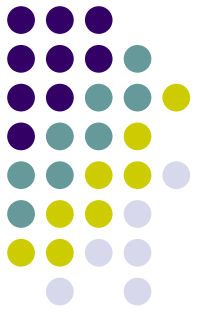
what:

r = read

w = write

x = execute

+ 加权限



2.4.2 chown

可以改用户/组

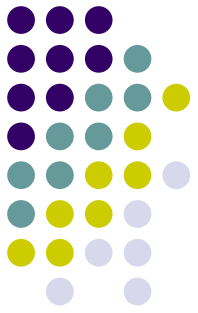
- 命令原意: change the owner of file
- 命令功能: 变更文件或目录的所有者 (用户/用户组)
- 命令格式: chown 用户 文件或目录
- 示例: chown root file

chown -R root dir

参数	说明
-R	递归处理，包括所有文件及子目录

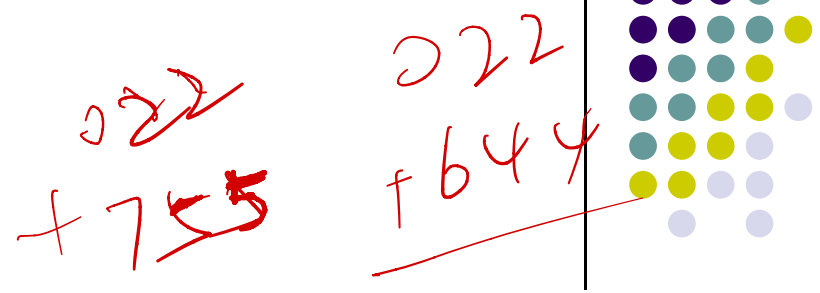
2.4.3 chgrp

更改所属组
只能改组



- 命令原意: change file group
- 命令功能: 变更文件或目录的所有者
- 命令格式: chgrp 组名 文件或目录名
- 示例: **chgrp** root file

2.4.4 umask

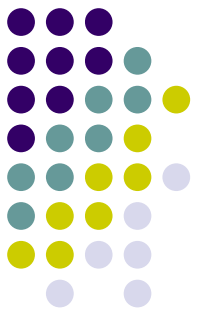


- 命令功能：设置或修改文件默认权限
- 命令格式：umask [mask]

注意：系统默认root屏蔽的权限为022（普通用户umask的默认值为002），因此新创建的目录权限就为 $777 - 022 = 755$ （普通用户为 $777 - 002 = 775$ ），新创建的普通文件权限为 $666 - 022 = 644$ （普通用户为 $666 - 002 = 664$ ）。

目录： $777 - 022 / 002$
755 775

文件： $666 - 022 / 002$
644 664



2.4.5 文件特殊权限

- 文件特殊权限

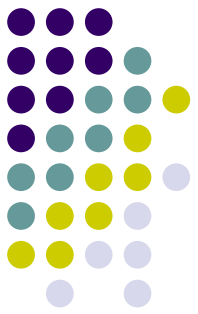
文件特殊权限有以下三种：

1) SUID: Set UID : s出现在文件所有者的x权限上

2) SGID: Set GID: s出现在用户组的x权限上

3) SBIT: Sticky Bit: t出现在目录的x权限上

other



2.4.5 文件特殊权限

- 文件特殊权限-SUID

1) **SUID: Set UID** 让普通用户临时拥有文件所有者的身份 (前提是该文件为可执行的二进制文件)

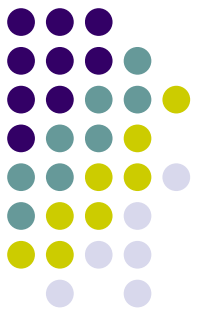
例如: `ll /usr/bin/passwd`

注意: 当用户执行passwd命令的时候, 需要修改

/etc/shadow文件, 而该文件普通用户并没有任何权限。

修改了SUID, 同时有x权限. | 有r, 没有x等于读的S

```
mylinux@ubuntu:~$ ll /usr/bin/passwd
-rwsr-xr-x 1 root root 68208 Jul 14 2021 /usr/bin/passwd*
mylinux@ubuntu:~$ ll /etc/shadow
-rw-r----- 1 root shadow 1417 Jul 29 18:18 /etc/shadow
```



2.4.5 文件特殊权限

- 文件特殊权限-SGID

2) **SGID: Set GID** 作用于文件时, 让普通用户临时拥有该文件所属组下用户的身份; 作用于目录时, 任何用户在该目录下新建的目录或文件所属组跟该目录是一样的

例如: ll /usr/bin/mlocate

26254: 组用户身份
解释: 26254 26254 / 组

当用户执行mlocate命令的时候, 需要读取
/var/lib/mlocate/mlocate.db文件。

有同样的组用户
身份

```
mylinux@ubuntu:~$ ll /usr/bin/mlocate
-rwxr-sr-x 1 root mlocate 47344 Jul 16 2019 /usr/bin/mlocate*
mylinux@ubuntu:~$ ll /var/lib/mlocate/mlocate.db
-rw-r----- 1 root mlocate 11004244 Oct 29 00:46 /var/lib/mlocate/mlocate.db
```




2.4.5 文件特殊权限

- 文件特殊权限-SBIT

3) **SBIT: Sticky Bit** (只对目录有效) 防止其他用户删除自己的文件, root用户除外

例如: `ls -dl /tmp` (当用户在该目录下创建文件或目录时, 仅由自己与root才有权利删除该文件)

```
mylinux@ubuntu:~$ ll -d /tmp
drwxrwxrwt 21 root root 4096 Oct 29 00:42 /tmp/
```



2.4.5 文件特殊权限

• 文件特殊权限

文件特殊权限的数字表示法

- 当文件具有特殊权限SUID时，其权限的数字表示法需要在原来的三位数字前面加上4；

-rwsr-xr-x ⇒ 4655-

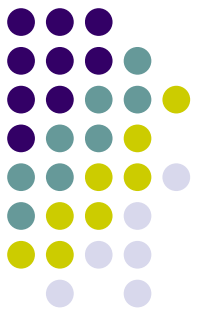
数字4 = 有x权限
-rwsr-xr-x. 1 root root 27856 Apr 1 2020 /bin/passwd 4755

- 当文件具有特殊权限SGID时，其权限的数字表示法需要在原来的三位数字前面加上2；

-rwx--s--x. 1 root slocate 40520 Apr 11 2018 /bin/locate 2711

- 当文件具有特殊权限SBIT时，其权限的数字表示法需要在原来的三位数字前面加上1。

drwxrwxrwt. 32 root root 4096 Jun 4 16:00 /tmp 1777



2.4.5 文件特殊权限

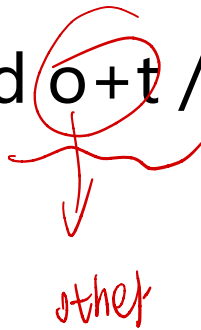
- 文件特殊权限

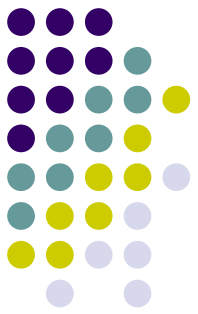
文件特殊权限修改

`chmod u+s /usr/bin/l` $\implies$ `chmod 4755 /usr/bin/l`

`chmod g+s /usr/bin/l` $\implies$ `chmod 2755 /usr/bin/l`

`chmod o+t /var/mydir` $\implies$ `chmod 1755 /var/mydir`


other

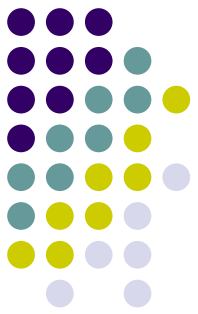


2.4.5 文件特殊权限

- 文件特殊权限

文件特殊权限修改

```
mylinux@ubuntu:~$ ls /root
ls: cannot open directory '/root': Permission denied
mylinux@ubuntu:~$ sudo chmod u+s /usr/bin/ls
mylinux@ubuntu:~$ ls /root
snap
mylinux@ubuntu:~$ ll /usr/bin/ls
-rwsr-xr-x 1 root root 142144 Sep  5  2019 /usr/bin/ls*
mylinux@ubuntu:~$ sudo chmod u-s /usr/bin/ls
mylinux@ubuntu:~$ ll /usr/bin/ls
-rwxr-xr-x 1 root root 142144 Sep  5  2019 /usr/bin/ls*
```



2.4.5 文件特殊权限

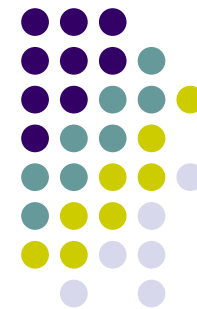
- 文件特殊权限

文件特殊权限修改

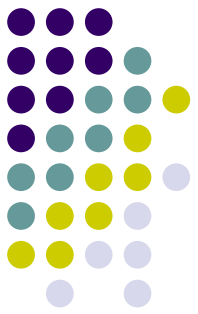
664

```
mylinux@ubuntu:~$ touch m1 m2 m3
mylinux@ubuntu:~$ ll -ld m*
-rw-rw-r-- 1 mylinux mylinux 0 Oct 29 01:05 m1
-rw-rw-r-- 1 mylinux mylinux 0 Oct 29 01:05 m2
-rw-rw-r-- 1 mylinux mylinux 0 Oct 29 01:05 m3
drwxrwxr-x 3 mylinux mylinux 4096 Sep 14 23:21 mydir/
mylinux@ubuntu:~$ chmod u+s m1
mylinux@ubuntu:~$ chmod u+sx m2
mylinux@ubuntu:~$ chmod 1664 m3
mylinux@ubuntu:~$ ll -ld m*
-rwSrw-r-- 1 mylinux mylinux 0 Oct 29 01:05 m1
-rwsrw-r-- 1 mylinux mylinux 0 Oct 29 01:05 m2*
-rw-rw-r-T 1 mylinux mylinux 0 Oct 29 01:05 m3
drwxrwxr-x 3 mylinux mylinux 4096 Sep 14 23:21 mydir/
```

2.5 文件压缩解压缩

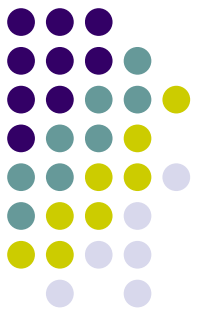


- tar
- zip/unzip
- gzip/gunzip
- bzip2/bunzip2



2.5 文件压缩解压

- Linux系统中压缩包的后缀有.zip、.gz、.bz2、.xz等，后缀不同表示压缩方式不同，使用的压缩命令也不同。压缩与解压成对使用，生成上述几种形式的压缩包所用的命令分别为zip、gzip、bzip2、xz，解压缩时使用的命令分别为unzip、gunzip、bzip2、xz。此外，Linux系统中提供了专门用于打包文件的命令——tar。



2.5.1 tar

- 命令功能：打包多个目录或文件，通常与压缩命令一起使用
- 命令格式：tar [选项] 目录

选项	说明
-c	产生.tar打包文件
-v	打包时显示详细信息
-f	指定压缩后的文件名
-z	有gz属性的软件包
-x	从打包文件中还原文件。
-C	指定解压的目标目录
-j	有bz2属性的软件包



2.5.1 tar

示例:

- #将目录test下的文件打包

```
tar -cvf test.tar ./test
```

- #将目录test下的文件打包, 并以gzip命令将包压缩

```
tar -zcvf test.tar.gz ./test
```

- #将目录test下的文件打包, 并以bzip2命令将包压缩

```
tar -jcvf test.tar.bz2 ./test
```

- #从包test.tar.bz2中还原文件

```
tar -xvf test.tar.bz2
```

没有选项 ⇒ 解压到当前目录下。



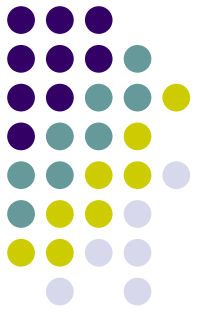
2.5.2 zip/unzip

- 命令功能：压缩文件，获得.zip格式的压缩包，压缩后保存源文件，有的系统默认没有安装
- 命令格式：zip [-r] [压缩后文件名称] 文件或目录

选项	说明
-j	只保留文件名称及其内容，不存放任何目录名称
-m	文件压缩完成后，删除原始文件
-o	以压缩文件内拥有最新更改时间的文件为准，更新压缩文件的更改时间
-r	当参数为目录时，递归处理目录下的所有文件和子目录

示例：

- #将目录test下的文件压缩
zip -r test.zip ./test



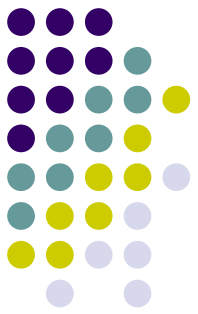
2.5.2 zip/unzip

- 命令功能：解压文件
- 命令格式：unzip [选项] 压缩包包名

选项	说明
-l	显示指定压缩包中所包含的文件
-M	将输出结果送到more程序处理
-n	解压时不要覆盖原有文件
-o	命令执行后直接覆盖原有文件
-d	指定解压后文件要存放的目录

示例：

- #将压缩文件test.zip解压到test1目录下
unzip test.zip -d test1

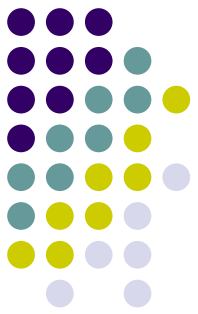


2.5.3 gzip/gunzip

- 命令功能：压缩文件，获得.gz格式的压缩包，压缩后不保存源文件，若同时列出多个文件，则每个文件会被单独压缩。
- 命令格式：`gzip [选项] 文件`
`gunzip [选项] 压缩包包名`

示例：

- #将目录test中的文件与所有子目录文件压缩成.gz文件
`gzip -r test/*`
- #将压缩文件test.gz解压
`gunzip test.gz`



2.5.4 bzip2/bunzip2

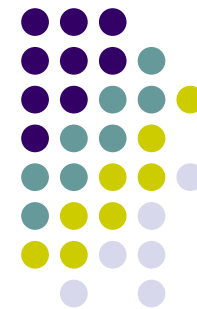
- 命令功能：压缩文件，获得.bz2格式的压缩包，使用选项 -k时保留源文件，有的系统默认没有安装。
- 命令格式：bzip2 [选项] 文件或目录
bunzip2 [选项] 压缩包包名
- 示例：

```
# bzip2 a.txt
```

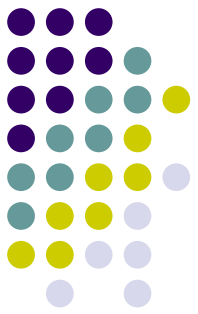
```
# bzip2 -d a.txt.bz2
```

```
# bunzip2 a.txt.bz2
```

Part 3 用户管理

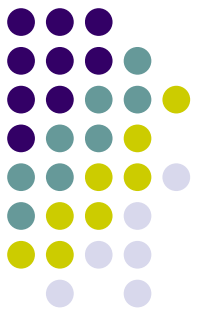


- Linux用户
- 用户管理
- Linux用户组
- 用户组管理
- 用户切换



3.1 Linux用户

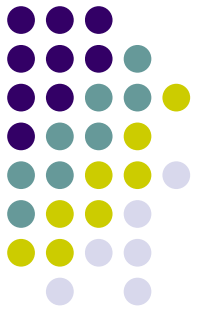
- Linux是一个多用户、多任务的分时操作系统，在一台Linux主机上，可同时登录多名用户
- Linux使用用户权限机制对系统进行管理
- 主要功能：提供不同用户使用本系统的权限分配
- 目的：保证用户数据与文件的安全。
- 账号实质：是一个用户在系统上的标识，系统依据账户来区分每个用户的文件、进程、任务，给每个用户提供特定的工作环境。



3.1 Linux用户

Linux操作系统用户的特点：

- 每个用户拥有一个**UserID**，操作系统实际读取的是UID；
- 每个用户属于一个**主组**，属于一个或多个**附属组**，一个用户最多有31个附属组；
- 每个组拥有一个**GroupID**；
- 每个进程以一个用户身份运行，该用户可对进程拥有资源控制权限；
- 每个可登陆用户拥有一个指定的Shell环境。



3.1 Linux用户

用户分类

超级用户root (UID=0)

- 每个Linux系统都必须有，并且只有一个；
- 拥有最高的权限，可以删除、终止任何程序。

系统用户 ($0 < \text{UID} < 1000/500$)

- 与系统运行和系统提供的服务密切相关的用户；
- 通常在安装相关的软件包时自动创建并保持默认状态；
- 系统用户不能登录计算机。

普通用户 ($\text{UID} \geq 1000/500$)

- 在系统安装后由超级用户创建的；
- 通常完成指定权限的操作，而且也只能操作自己所拥有权限的文件和目录。



3.1 Linux用户

- 涉及到的相关配置文件:

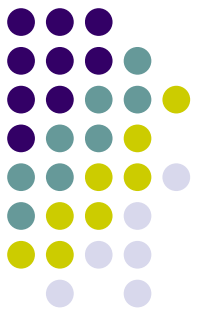
/etc/passwd 保存用户信息 每个用户对应一行

/etc/shadow 保存用户密码 (以加密形式保存)

/etc/group 保存组信息

/etc/login.defs 用户属性限制, 密码过期时间, 密码最大长度等限制

/etc/default/useradd 显示或更改默认的用户add配置文件



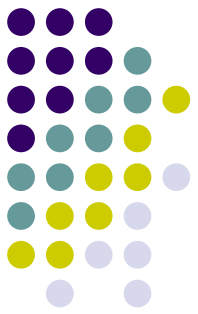
3.1 Linux用户

- 涉及到的相关配置文件：

1. /etc/passwd文件：记录了Linux系统中每一个用户的基本属性，对所有用户可读，每一行对应一个用户，用冒号分割，具体含义：

格式： username:password:uid:gid:userinfo:home:shell

- **用户名**：在系统中是唯一的，可由字母、数字和符号组成；
- **口令**：此字段用“x”代替，^{加密了}口令保存在/etc/shadow文件中。
- **用户ID (UID)**：系统内部用它来标识用户且唯一。
- **组ID (GID)**：系统内部用它来标识组属性。



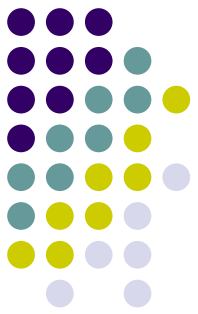
3.1 Linux用户

- 涉及到的相关配置文件：

1. /etc/passwd文件：

- 用户相关信息：例如用户全名等。
- 用户主目录：用户登录系统后所进入的目录。
- 用户登录环境：用户第一次登录Shell环境。

```
mylinux@ubuntu:~$ tail /etc/passwd
geoclue:x:122:127::/var/lib/geoclue:/usr/sbin/nologin
pulse:x:123:128:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
gnome-initial-setup:x:124:65534::/run/gnome-initial-setup:/bin/false
gdm:x:125:130:Gnome Display Manager:/var/lib/gdm3:/bin/false
sssd:x:126:131:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin
mylinux:x:1000:1000:mylinux,,,:/home/mylinux:/bin/bash
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
sshd:x:127:65534::/run/sshd:/usr/sbin/nologin
user1:x:1001:1001::/home/user1:/bin/sh
user2:x:1002:1002::/home/user2:/bin/sh
```



3.1 Linux用户

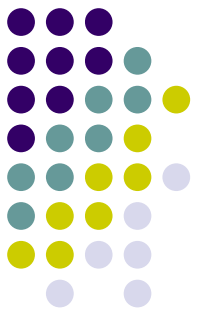
- 涉及到的相关配置文件：*其他用户无权限*

2. /etc/shadow文件：只有root用户有读权限

格式：

上次修改时间
username:password:lastchg:min:max:warn:inactive:expire:flag

- 1、**用户名**：用户登录到系统时使用的名字，而且是惟一的。
- 2、**口令**：存放MD5加密过的口令。
- 3、标识从1970年1月1日到用户最近一次修改口令所经过的天数；
- 4、口令保持有效的最小天数，即口令在多少天内不能被用户修改。
- 5、口令在多少天后需要被修改。



3.1 Linux用户

- 涉及到的相关配置文件：

2./etc/shadow文件

格式：

username:password:lastchg:min:max:warn:inactive:expire:flag

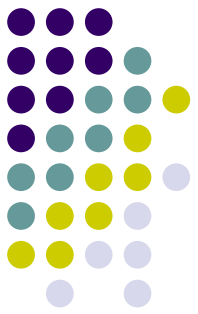
warn: 6、口令到期前多少天内给用户发出警告。

7、口令过期多少天后口令失效。

expire: 8、帐号失效日。

若失效日期设为2018年1月1日，则该字段为1970年1月1日到2018年1月1日的天数： $365 \times (2018 - 1970) + (2018 - 1970) / 4 + 1 = 17532$

9、保留域。

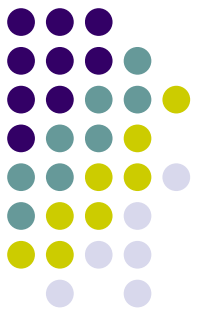


3.1 Linux用户

- 涉及到的相关配置文件:

2./etc/shadow文件

```
mylinux@ubuntu:~$ sudo head /etc/shadow
[sudo] password for mylinux:
root:!:19203:0:99999:7:::
daemon:*:19046:0:99999:7:::
bin:*:19046:0:99999:7:::
sys:*:19046:0:99999:7:::
sync:*:19046:0:99999:7:::
games:*:19046:0:99999:7:::
man:*:19046:0:99999:7:::
lp:*:19046:0:99999:7:::
mail:*:19046:0:99999:7:::
news:*:19046:0:99999:7:::
```



3.1 Linux用户

- 涉及到的相关配置文件：

3.用户模板文件目录/etc/skel

- .bash_profile或.profile: 每次登录时执行，定义初始化变量
- .bashrc: 每次开启新的终端时执行，定义永久的别名
- .bash_logout: 每次注销时执行

```
mylinux@ubuntu:~$ ls -a /etc/skel  
.  ..  .bash_logout  .bashrc  .profile
```



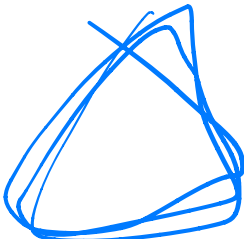

3.1 Linux用户

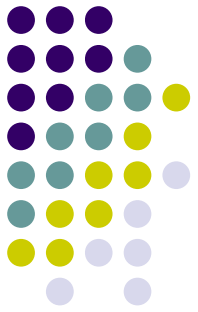
- 涉及到的相关配置文件:

4. /etc/default/useradd文件: 定义了新建用户的一些默认属性



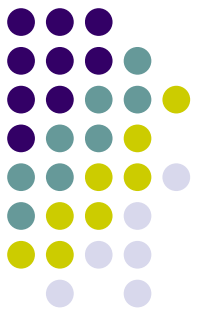
```
# useradd defaults file
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
CREATE_MAIL_SPOOL=yes
```





3.2 用户管理

- 添加用户 `useradd`
- 用户密码设置 `passwd`
- 删除用户 `userdel`
- 修改用户信息 `usermod`

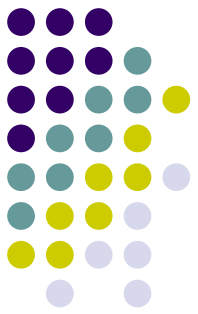


3.2.1 添加用户

- **useradd命令**：添加用户，命令的格式如下：

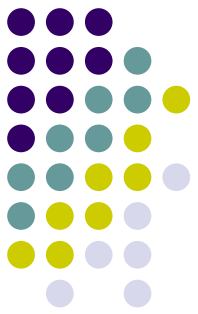
useradd [选项] 用户名

选项	说明
-d	指定用户登入时的目录
-c	指定账户的备注文字 <i>command</i>
-e	指定账号的有效期限 <i>expire</i>
-f	缓冲天数，密码过期时在指定天数后关闭该账号
-g	<u>指定用户所属组</u> <i>(主组)</i>
-G	<u>指定用户所属的附加用户组</u> <i>(附属组)</i>
-m	自动建立用户的登入目录
-r	创建系统账号 <i>1-999</i>
<u>-s</u>	<u>指定用户的登陆shell</u> <i>bin/bash</i>
-u	<u>指定用户的用户ID</u> 。若添加-o选项，则用户ID可与其它用户重复



3.2.1 添加用户

- **useradd user1** 命令默认创建用户 **user1**，创建过程：
 - 读取/etc/default/useradd，根据配置文件执行创建操作；
 - 在/etc/passwd文件中添加用户信息；
 - 如使用passwd命令创建密码，密码会被加密保存在/etc/shadow中；
 - 为user1创建家目录：/home/user1； (- m)
 - 将/etc/skel中的.bash开头的文件复制至/home/user1家目录；
 - 创建与用户名相同的user1组，user1用户默认属于user1同名组；
 - user1组信息保存在/etc/group配置文件中。

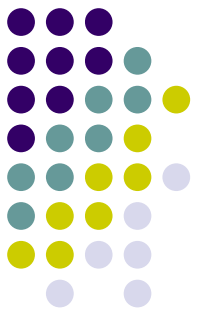


3.2.1 添加用户

- 添加用户举例

```
mylinux@ubuntu:~$ sudo useradd user01
mylinux@ubuntu:~$ tail -5 /etc/passwd
sssd:x:126:131:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin
mylinux:x:1000:1000:mylinux,,,:/home/mylinux:/bin/bash
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
sshd:x:127:65534:/:/run/sshd:/usr/sbin/nologin
user01:x:1001:1001:/:/home/user01:/bin/sh
mylinux@ubuntu:~$ su user01
Password:
su: Authentication failure
mylinux@ubuntu:~$ ls -l /home/
total 4
drwxr-xr-x 22 mylinux mylinux 4096 Nov  9 15:53 mylinux
```

创.
没有取用, 子建



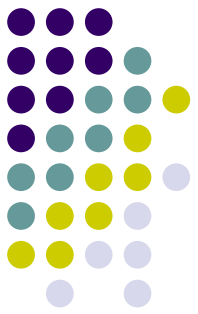
3.2.1 添加用户

- 添加用户 (Ubuntu)

默认创建的用户没有创建家目录，没有指定shell，创建用户时需要指定并设置密码后才能登录系统。

```
mylinux@ubuntu:~$ sudo useradd user01
mylinux@ubuntu:~$ tail -1 /etc/passwd
user01:x:1004:1005:~/home/user01:/bin/sh
mylinux@ubuntu:~$ ls /home
mylinux
mylinux@ubuntu:~$ sudo userdel user01
mylinux@ubuntu:~$ sudo useradd user01 -m -s /bin/bash
mylinux@ubuntu:~$ ls /home
mylinux user01
```

需要指定shell



3.2.1 添加用户

- 添加用户举例——**创建一个系统用户**

```
mylinux@ubuntu:~$ sudo useradd -r sysuser01
mylinux@ubuntu:~$ tail -2 /etc/passwd
user01:x:1001:1001:~/home/user01:/bin/sh
sysuser01:x:998:998:~/home/sysuser01:/bin/sh
```

→ close



3.2.1 添加用户

- 添加用户举例——创建新用户，指定用户账号有效期为1天

```
mylinux@ubuntu:~$ sudo useradd -c 1天过期 -e 1 user02
mylinux@ubuntu:~$ tail -2 /etc/passwd
sysuser01:x:998:998:~/home/sysuser01:/bin/sh
user02:x:1002:1002:1天过期:/home/user02:/bin/sh
```

```
mylinux@ubuntu:~$ sudo tail -1 /etc/shadow
user02:!:19306:0:99999:7::1:
```




3.2.2 用户密码设置

- **passwd命令**：用于设置用户的认证信息，包括用户密码、密码有效期等，命令的格式如下：

passwd [选项] 用户名

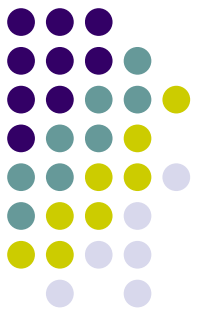
选项	说明
-l	锁定密码，锁定后密码失效，无法登陆（新用户默认锁定）
-d	删除密码，仅系统管理员可使用
-S	列出密码相关信息，仅系统管理员可使用
-f	强制执行



3.2.2 用户密码设置

- 为用户设置密码

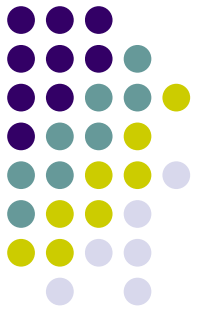
```
mylinux@ubuntu:~$ sudo tail -3 /etc/shadow
user01:!:19306:0:99999:7:::
sysuser01:!:19306:0:99999:7:::
user02:!:19306:0:99999:7::1:
mylinux@ubuntu:~$ sudo passwd user01
New password:
Retype new password:
passwd: password updated successfully
mylinux@ubuntu:~$ sudo tail -3 /etc/shadow
user01:$6$T1SU6Wo3k93T6nJK$n7CtZPY0v08DFZG/wZSIQHxVxP6UnHLf2N05p5DgTfxFHIdbM2YwYR6D3701Q23tANsb6McMV
CBJdhX9oaJhx/:19307:0:99999:7:::
sysuser01:!:19306:0:99999:7:::
user02:!:19306:0:99999:7::1:
```



3.2.2 用户密码设置

- 说明：

- ①root用户可随意修改密码，即便出现警告，也可成功保存。
- ②普通用户在设置密码时应尽量复杂，避免与用户名相同。
- ③修改当前登陆账户的密码时，可缺省用户名。
- ④用户密码保存在文件/etc/shadow中。



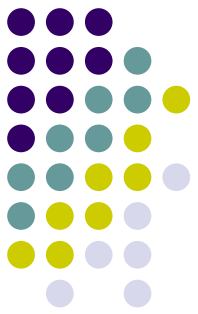
3.2.3 删除用户

- userdel命令: 可删除指定账户以及与账户相关的文件和信息。命令格式如下:

userdel [选项] 用户名

选项	说明
-f	强制删除用户，即便该用户为当前用户
-r	删除用户的同时，删除与用户相关的所有文件

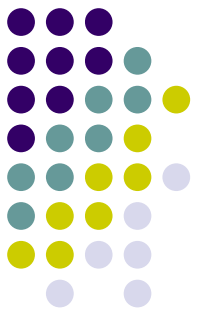
删除有关的文件.



3.2.3 删除用户

- 示例: `userdel -r user03`

```
mylinux@ubuntu:~$ sudo tail -3 /etc/passwd
sysuser01:x:998:998:~/home/sysuser01:/bin/sh
user02:x:1002:1002:1天过期:/home/user02:/bin/sh
user03:x:1004:1003:~/home/user01:/bin/sh
mylinux@ubuntu:~$ sudo userdel -r user03
userdel: user03 mail spool (/var/mail/user03) not found
mylinux@ubuntu:~$ sudo tail -3 /etc/passwd
sshd:x:127:65534:~/run/sshd:/usr/sbin/nologin
sysuser01:x:998:998:~/home/sysuser01:/bin/sh
user02:x:1002:1002:1天过期:/home/user02:/bin/sh
```

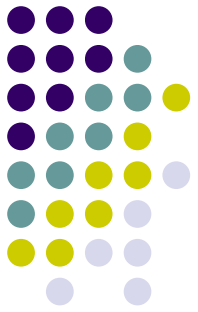


3.2.4 修改用户信息

- **usermod命令**：修改用户账号信息，即修改账号的属性，如用户ID、主目录、用户组、登陆shell等。修改用户账号信息的命令为usermod，其命令格式如下：

usermod 选项 用户名





3.2.4 修改用户信息

选项	说明
<u>-c</u>	<u>修改用户账号的备注信息</u>
<u>-d</u>	<u>修改用户的登入目录</u>
-e	修改账号的有效期限 <i>expire</i>
-f	修改缓冲天数，即修改密码过期后关闭账号的时间
<u>-g</u>	<u>修改用户所属组</u>
<u>-l</u> <i>login</i>	<u>修改用户账号名称</u>
<u>-L</u> <i>Lock</i>	<u>锁定用户密码，使密码失效</u>
<u>-s</u>	<u>修改用户登陆后使用的shell</u>
<u>-u</u>	<u>修改用户ID</u>
<u>-U</u>	<u>解除密码锁定</u>



3.2.4 修改用户信息

- 示例：为用户修改用户ID

```
mylinux@ubuntu:~$ sudo tail -3 /etc/passwd
[sudo] password for mylinux:
sysuser01:x:998:998:~/home/sysuser01:/bin/sh
user02:x:1002:1002:1天过期:/home/user02:/bin/sh
user01:x:1003:1003:~/home/user01:/bin/sh
mylinux@ubuntu:~$ sudo usermod -u 1004 user01
mylinux@ubuntu:~$ sudo tail -3 /etc/passwd
sysuser01:x:998:998:~/home/sysuser01:/bin/sh
user02:x:1002:1002:1天过期:/home/user02:/bin/sh
user01:x:1004:1003:~/home/user01:/bin/sh
```




3.2.4 修改用户信息

- 示例：为用户修改用户名

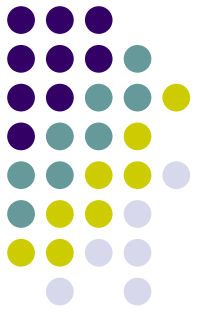
```
mylinux@ubuntu:~$ sudo usermod user01 -l user03
mylinux@ubuntu:~$ sudo tail -3 /etc/passwd
sysuser01:x:998:998::/home/sysuser01:/bin/sh
user02:x:1002:1002:1天过期:/home/user02:/bin/sh
user03:x:1004:1003::/home/user01:/bin/sh
```



3.2.5 管理有效期

chage命令：修改帐号和密码的有效期限

- **格式**：chage [选项] <用户名>
- **选项**：
 - m：密码可更改的最小天数。为零时代表任何时候都可以更改密码。
 - M：密码保持有效的最大天数。
 - W：用户密码到期前，提前收到警告信息的天数。
 - E：帐号到期的日期。到期后此帐号将不可用。-1表示永不过期，0表示马上过期



3.2.5 管理有效期

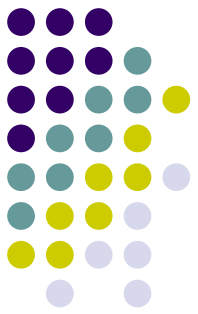
chage命令:

选项:

-d: 上一次更改的日期。

~~-I~~ ^{大写i} -I: 停滞时期。如果一个密码已过期这些天，那么此帐号将不可用。

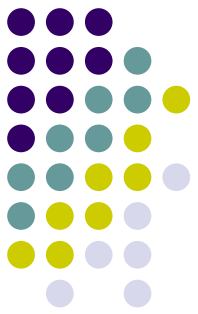
^{小写l} -l: 例出当前的设置。由非特权用户来确定他们的密码或帐号何时过期。



3.2.5 管理有效期

查看用户账户和密码有效期

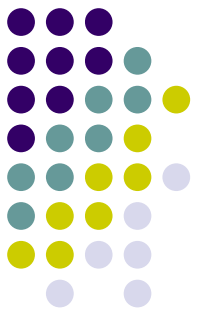
```
mylinux@ubuntu:~$ sudo chage user03 -l
Last password change                : Nov 11, 2022
Password expires                    : never
Password inactive                   : never
Account expires                    : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
mylinux@ubuntu:~$ sudo chage user02 -l
Last password change                : Nov 10, 2022
Password expires                    : never
Password inactive                   : never
Account expires                    : Jan 02, 1970
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
```



3.2.5 管理有效期

修改用户账户有效期信息

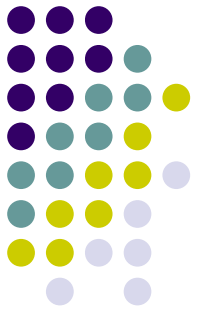
```
mylinux@ubuntu:~$ sudo chage user02 -l
Last password change                : Nov 10, 2022
Password expires                    : Dec 15, 2022
Password inactive                   : never
Account expires                    : Jan 01, 1970
Minimum number of days between password change : 6
Maximum number of days between password change : 35
Number of days of warning before password expires : 5
mylinux@ubuntu:~$ sudo chage user02 -E -1
mylinux@ubuntu:~$ sudo chage user02 -l
Last password change                : Nov 10, 2022
Password expires                    : Dec 15, 2022
Password inactive                   : never
Account expires                    : never
Minimum number of days between password change : 6
Maximum number of days between password change : 35
Number of days of warning before password expires : 5
```



3.2.5 管理有效期

修改用户账户有效期信息

```
mylinux@ubuntu:~$ sudo chage user02 -m 6 -M 35 -W 5
mylinux@ubuntu:~$ sudo chage user02 -l
Last password change                : Nov 10, 2022
Password expires                     : Dec 15, 2022
Password inactive                    : never
Account expires                     : Jan 02, 1970
Minimum number of days between password change : 6
Maximum number of days between password change : 35
Number of days of warning before password expires : 5
```



3.2.6 ACL机制

- ACL (Access Control List) 存取控制列表 可以针对单一用户或组来设置特定的权限

更精细



3.2.6 ACL机制

ACL 相关命令

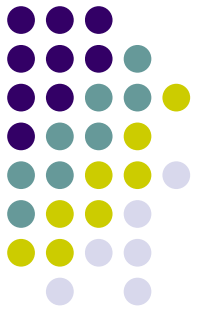
- setfacl: 设置文件的ACL

chmod根据文件所有者、所属群组及其他用户三级权限进行权限分配，而setfacl可以针对每一文件或目录进行更加精准的权限分配。

- getfacl: 获取文件的ACL
- chacl: 更改文件或目录的ACL

与chmod相似，但是更为强大精细，通过chmod可以控制文件被何人调用，但若是某一用户的文件只想给特定的用户看时，则需要

chacl完成用户的需求。



3.2.6 ACL机制

setfacl命令:

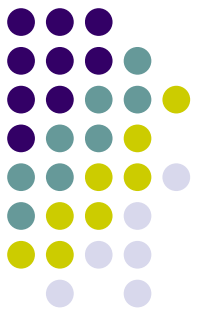
- **格式:** setfacl 选项 文件名
- **功能:** 设定用户或群组对指定文件的访问权限。
- **选项:**
 - m 修改指定文件的acl, 不能和-x混合使用
 - x 删除后续参数
 - b 删除所有acl设定参数
 - k 移除预设的acl参数
 - R 递归设置acl参数
 - d 预设目录的acl参数



3.2.6 ACL机制

getfacl命令:

- **格式:** getfacl 选项 文件名
- **功能:** 查看文件或目录当前设定的 ACL 权限信息。
- **选项:**
 - a: 仅显示文件访问控制列表
 - d: 仅显示默认的访问控制列表
 - c: 不显示注释表头
 - e: 显示所有有效权限
 - E: 显示无效权限
 - s: 跳过只有基条目(base entries)的文件



3.2.6 ACL机制

- **setfacl命令:**

为文件/home/mylinux/file添加用户名为stu01的用户的写权限。

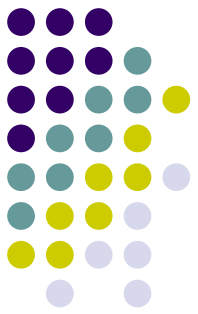
```
mylinux@ubuntu:~$ ls -l file
-rw-rw-r--+ 1 mylinux mylinux 10 Nov 11 04:55 file
mylinux@ubuntu:~$ su - stu01
Password:
stu01@ubuntu:~$ echo hello >/home/mylinux/file
-bash: /home/mylinux/file: Permission denied
```

```
mylinux@ubuntu:~$ sudo setfacl -m u:stu01:rw- /home/mylinux/file
[sudo] password for mylinux:
mylinux@ubuntu:~$ su - stu01
Password:
stu01@ubuntu:~$ echo hello acl >/home/mylinux/file
stu01@ubuntu:~$ cat /home/mylinux/file
hello acl
```

Handwritten notes in red:
- Under `setfacl`: 设置权限
- Under `-m u:stu01:rw-`: 设置权限
- Under `acl` in the file content: 增加了acl权限

```
stu01@ubuntu:~$ ls -l /home/mylinux/file
-rw-rw-r--+ 1 mylinux mylinux 10 Nov 11 04:55 /home/mylinux/file
```

Handwritten note in red:
- Under the new permissions `-rw-rw-r--+`: 增加了acl权限

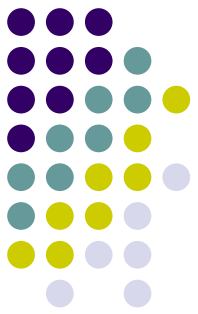


3.2.6 ACL机制

- **getfacl命令:**

查看文件file的权限: `getfacl /home/mylinux/file`。

```
stu01@ubuntu:~$ getfacl /home/mylinux/file
getfacl: Removing leading '/' from absolute path names
# file: home/mylinux/file
# owner: mylinux
# group: mylinux
user::rw-
user:stu01:rw-
group::rw-
mask::rw-
other::r--
```



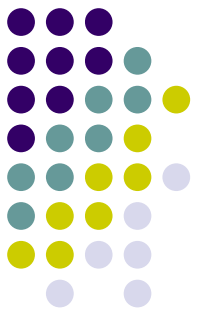
3.2.6 ACL机制

- **setfacl命令:**

为文件/home/mylinux/file添加用户组group01用户的读写权限。

```
mylinux@ubuntu:~$ getfacl file2
# file: file2
# owner: mylinux
# group: mylinux
user::rw-
group::rw-
other::r--

mylinux@ubuntu:~$ sudo setfacl -m g:group01:rw- /home/mylinux/file2
mylinux@ubuntu:~$ getfacl file2
# file: file2
# owner: mylinux
# group: mylinux
user::rw-
group::rw-
group:group01:rw-
mask::rw-
other::r--
```



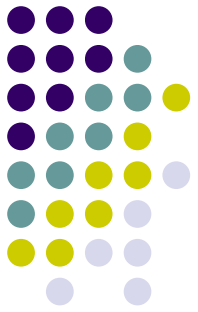
3.2.6 ACL机制

- **setfacl命令:**

为文件/home/mylinux/file删除用户组group01用户的读写权限。

```
mylinux@ubuntu:~$ getfacl file2
# file: file2
# owner: mylinux
# group: mylinux
user::rw-
group::rw-
group:group01:rw-
mask::rw-
other::r--
```

```
mylinux@ubuntu:~$ sudo setfacl -x g:group01 /home/mylinux/file2
mylinux@ubuntu:~$ getfacl file2
# file: file2
# owner: mylinux
# group: mylinux
user::rw-
group::rw-
mask::rw-
other::r--
```



3.2.6 ACL机制

chacl命令:

- **格式:** chacl [选项] 文件
- **功能:** 更改文件或目录当前设定的 ACL 权限信息。
- **选项:**
 - b 同时修改文件权限和默认目录权限
 - d 设置目录的默认权限
 - R 只删除文件的权限
 - D 只删除目录的权限
 - B 删除所有权限
 - l 列出所有文件和目录权限
 - r 设置所有目录和子目录下的权限



3.2.6 ACL机制

清除文件名为file的文件上的ACL设置

```
mylinux@ubuntu:~$ getfacl file
# file: file
# owner: mylinux
# group: mylinux
user::rw-
group::rw-
group:group01:rw-
mask::rw-
other::r--
```

删除所有ACL权限

```
mylinux@ubuntu:~$ chacl -B file
mylinux@ubuntu:~$ getfacl file
# file: file
# owner: mylinux
# group: mylinux
user::rw-
group::rw-
other::r--
```



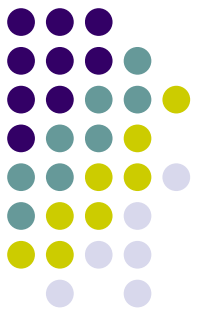

3.2.7 用户管理应用举例

- 添加一个用户myuser，指定主用户组为mygroup，附加用户组为addgroup，用户主目录为/opt/myusers



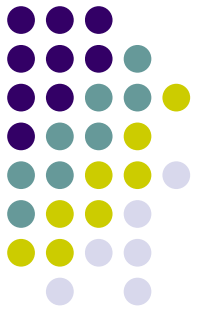
3.2.7 用户管理应用举例

- 添加一个用户testuser, 指定UID为686, 默认shell为/bin/csh, 主用户组为mygroup, 附加用户组为addgroup, 添加用户描述为: This is test user



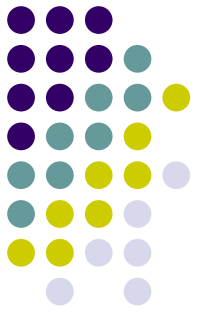
3.2.7 用户管理应用举例

- 修改用户testuser主用户组为testgroup, 附加用户组为testaddgroup, 默认登录shell为
/bin/bash



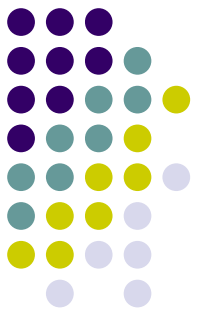
3.3 Linux用户组

- 添加用户组
- 删除用户组
- 修改用户组



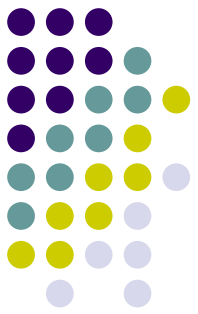
3.3 Linux用户组

- **用户组**
 - 组是具有相同特性的用户集合。
 - 设置目的：便于权限的统一组织和分配。
 - 注意：组账号不能登录计算机



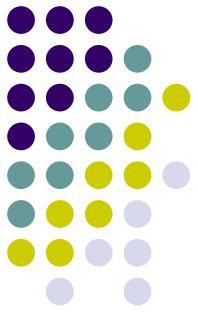
3.3 Linux用户组

- Linux组特点：
 - 每个组有一个组ID；
 - 组信息保存在/etc/group中；
 - 每个用户至少拥有一个主组，同时还可以拥有31个附属组。



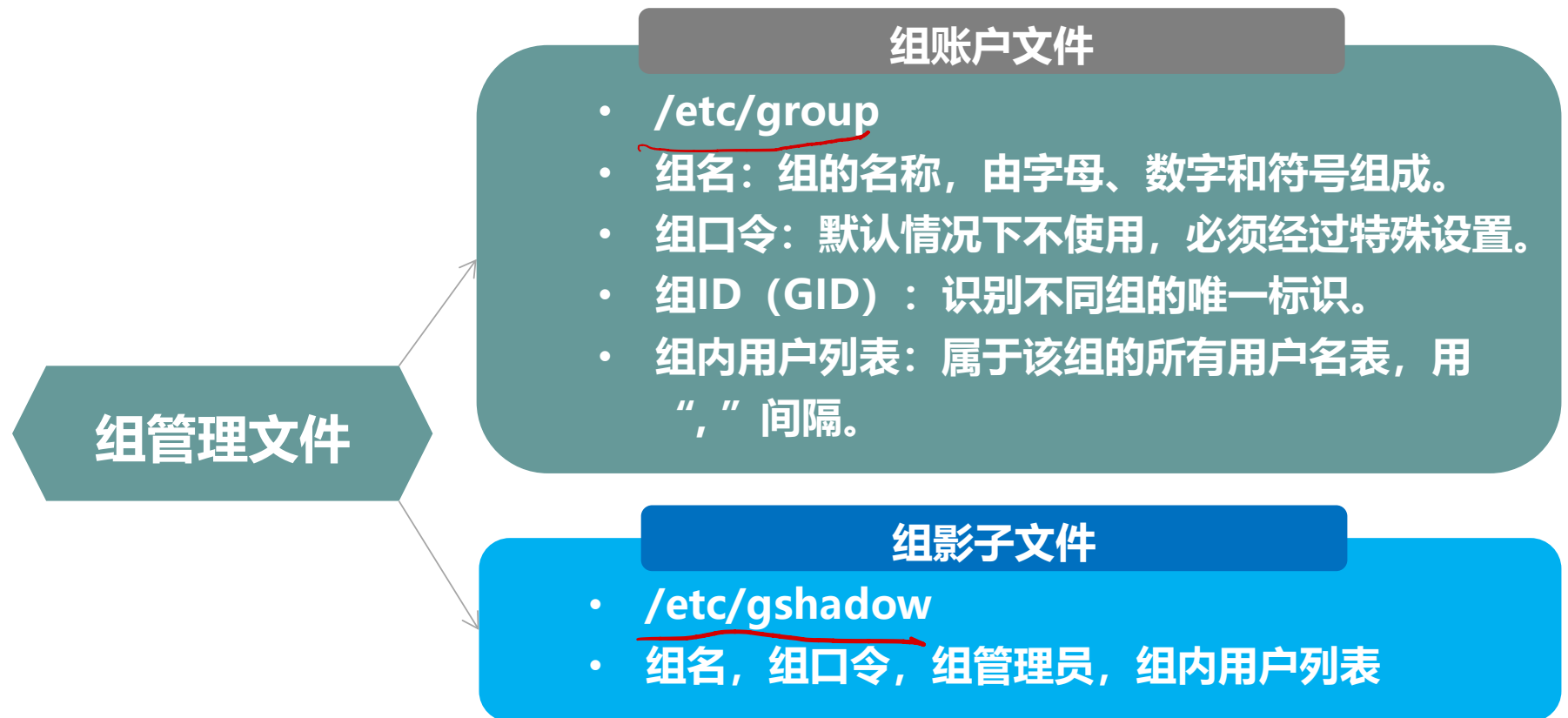
3.3 Linux用户组

- 用户与组属于多对多的关系。一个用户可以同时属于多个组。
- 组账户分为⁰超级组 (Superuser Group)、^{1~997}系统组 (System Group) 和^{2/1000}自定义组。GID分别为0、1~999、从1000开始。



3.3 Linux用户组

用户组配置文件





3.3 Linux用户组

用户组配置文件

组账户文件/etc/group组成

格式： group_name:group_password:group_id:group_members

组名： 在系统中是唯一的，可由字母、数字和符号组成；

口令： 此字段用 “x” 代替，口令保存在/etc/gshadow文件中。

组ID (GID)： 系统内部用它来标识用户组且唯一。默认用户组id从1000开始

组成员信息： 属于该组的用户清单，用逗号分割

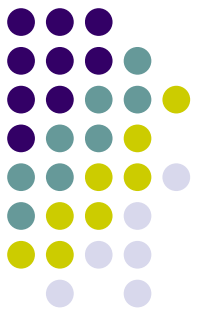


3.3 Linux用户组

用户组配置文件

组账户文件/etc/group组成

```
mylinux@ubuntu:~$ sudo tail -5 /etc/group
[sudo] password for mylinux:
systemd-coredump:x:999:
mlocate:x:134:
sysuser01:x:998:
user02:x:1002:
user01:x:1003:
```



3.3 Linux用户组

用户组配置文件

组影子文件/etc/gshadow组成

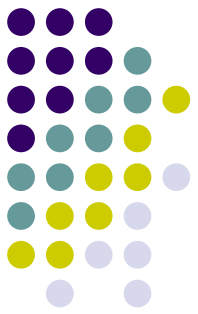
格式: group_name:group_password:group_id:group_members

组名

口令: 加密后的用户组密码。若显示为!，则表示该组没有密码。

组ID (GID)

组成员信息: 属于该组的用户清单，用逗号分割

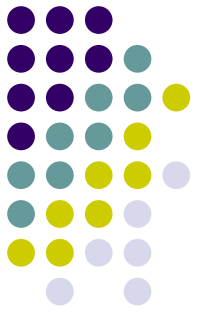


3.3 Linux用户组

用户组配置文件

组影子文件/etc/gshadow组成

```
mylinux@ubuntu:~$ sudo tail -5 /etc/gshadow
systemd-coredump:!!!:
mlocate:!!!:
sysuser01:!!!:
user02:!!!:
user01:!!!:
```



3.4 用户组管理

- 添加用户组
- 删除用户组
- 修改用户组



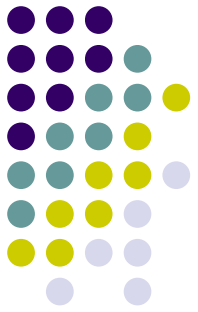
3.3.1 添加用户组

- 默认情况下新建用户的用户组与用户名相同，在创建用户的同时被创建。主动添加用户组时使用的命令为 `groupadd`，该命令的格式如下：

`groupadd [选项] 用户名`

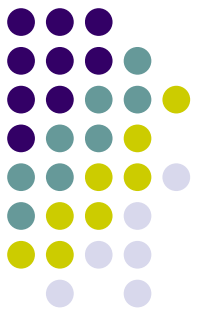
选项	说明
<code>-g</code>	指定新建用户组的组ID
<code>-r</code>	创建系统用户组，组ID取值范围为1~499
<code>-o</code>	允许创建组ID已存在的用户组

示例：`groupadd -g 550 group1`



3.3.2 删除用户组

- 若要删除已存在的用户组，可使用groupdel命令，该命令的格式如下：
- **groupdel** 参数
- 示例：**groupdel** group1



3.3.3 修改用户组属性

- 用户组的一些属性，如组ID和组名，都可以被修改。
修改用户组属性的命令为groupmod，其命令格式如下：

- groupmod** [选项] [用户组]

选项	说明
-g	为用户组指定新的组ID
-n	修改用户组的组名
-o	允许创建组ID已存在的用户组

- 示例：
groupadd group1 -g 550
groupmod -o group2 -g 550
groupmod group1 -n qust

password



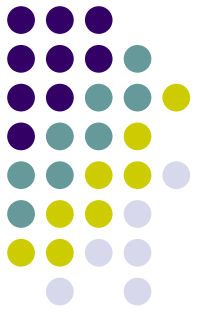
3.3.4 用户组切换

- 若用户被创建时没有指定用户组，系统会为用户创建一个与用户名相同的组，这个组就是基本组/主组；除基本组之外，用户所在的组都是附加组，为用户指定附加组，可以使用户拥有对应组的权限。
- **newgrp** [用户组]



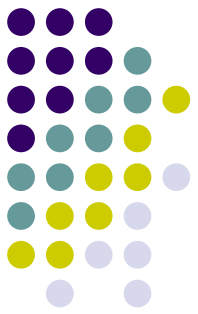
3.3 用户组管理举例

- 创建3个用户组group1, group2, group3
- 创建一个用户user1, 指定主用户组为group1, 附属用户组为group2, group3



3.4 用户切换

- 切换用户 su: switch user 所有用户
- sudo: super user do



3.4.1 su

- 使用su命令切换用户是最简单的用户切换方式，该命令可以在任意用户之间切换，其命令格式如下：

su user

还是使用原来用户的环境变量

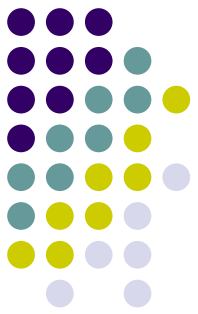
- su** - username

- 需要注意，以上的“-”为一个选项，类似于“-l”，选项与用户名之间应有一个空格。

su - user = su -l user

参数	说明
-c	执行完指定的指令后，切换回原来的用户
-l	切换用户的同时，切换到对应用户的工作目录，环境变量也会随之改变
-m, -p	切换用户时，不改变环境变量
-s	指定要执行的shell

command 临时切换用户身份，切换回原来的用户



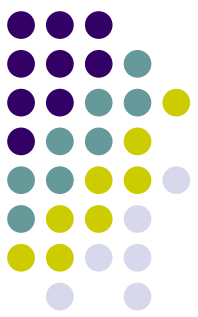
3.4.1 su

示例: su

```
itheima@localhost:/home/itcast
文件(F) 编辑(E) 查看(V) 搜索 (S) 终端(T) 帮助(H)
[ itheima@localhost ~]$ su
密码 :
[ root@localhost itcast]#
```

- 说明:

- ①使用su命令切换用户时，需要输入的是**目标用户**的密码。
- ②选项与用户名缺省的情况下，只切换用户，但不改变用户环境。
- ③由root用户切换到其它用户，可以不输入密码。
- ④在命令行输入“exit”，可退出目标用户。



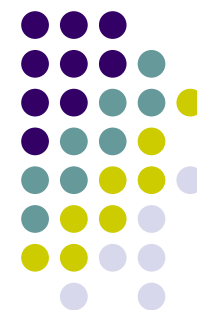
3.4.1 su

范例 1: 从普通用户 `stu01` 切换为 `root` 用户身份，同时带环境切换，然后从 `root` 切换为 `stu02` 用户，不带环境切换，如清单 2 清单 所示。

清单 2

```
#用 env 命令结合 grep 查看当前环境中的 PATH 变量值
$ env|grep "PATH"
PATH=/usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/stu01/.local/bin:/home/stu01/bin
#带环境切换为 root
$ su - root
密码:
#再次用 env 命令结合 grep 查看当前环境中的 PATH 变量值，发生了变化
# env|grep "PATH"
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin:/root/bin
#从 root 切换为普通用户无需输入密码
# su stu02
#不带环境切换后，用 env 命令结合 grep 查看当前环境中的 PATH 变量值，未发生变化
$ env|grep "PATH"
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin:/root/bin
```

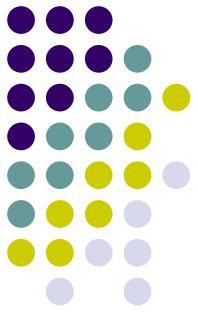
3.4.1 su



范例 2: 在 `user01` 用户身份下，用 `su` 命令带环境临时切换为 `stu` 用户身份执行命令，如清单 3 所示。

清单 3

```
$ whoami
user01
#以 stu 用户身份执行 touch newfile 命令，由于带环境切换，因此该 newfile 事实上创建在 stu 家目录中
#需要输入 stu 用户密码
$ su - stu -c 'touch newfile'
密码:
#以 stu 用户身份去查看刚刚创建的新文件
```



3.4.2 sudo

类似 su -c

- sudo命令的格式如下：
- **sudo** [选项] -u 用户 [命令]

选项	说明
-b	在后台执行命令
-h	显示帮助
-H	将HOME环境变量设置为新身份的HOME环境变量
-k	结束密码的有效期限
-l	列出目前用户可执行与无法执行的命令
-p	改变询问密码的提示符号
-s	执行指定的shell
-u	以指定的用户作为新的身份，即切换到指定用户。默认切换到root用户



3.4.2 sudo

sudo说明:

- ①sudo命令可以视为受限的su, 它可以使“部分”用户使用其它用户的身份执行命令。
- ②在使用sudo命令之前, 需要通过修改/etc/sudoers文件, 为当前用户配置要使用的权限。
- ③使用root用户执行visudo命令, 打开/etc/sudoers文件进行修改。

```
## Next comes the main part: which users can run what software on
## which machines (the sudoers file can be shared between multiple
## systems).
## Syntax:
##
##      user    MACHINE=COMMANDS
##
## The COMMANDS section may have other options added to it.
##
## Allow root to run any commands anywhere
root    ALL=(ALL)    ALL
```



3.4.2 sudo

④/etc/sudoers文件中的一条配置信息，该信息设置了root用户可能任何情境下执行任何命令。

root ALL=(ALL) ALL

语法格式：

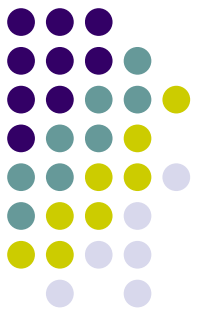
用户名 主机名(切换的用户名)可以执行的权限命令

例：user ALL(root)/bin/l

要求：可以执行的命令必须要绝对路径

允许此用户 在任意主机上获得所有权限

```
## Allow root to run any commands anywhere
root    ALL=(ALL)    ALL
#
## Allow user to run cat commands anywhere
user    ALL=(root)/bin/cat
```



3.4.2 sudo

- /etc/sudoers文件设置举例

```
# Host alias specification
Host_Alias    EPG = 192.168.1.1, 192.168.1.2

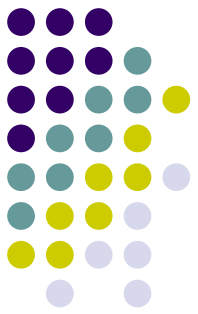
# User alias specification

# Cmnd alias specification
Cmnd_Alias    SQUID = /opt/vtbin/squid_refresh, /sbin/service, /bin/rm

# Defaults specification

# User privilege specification
root  ALL=(ALL) ALL
support EPG=(ALL) NOPASSWD: SQUID

# Uncomment to allow people in group wheel to run all commands
# %wheel    ALL=(ALL)    ALL
```



3.4.2 sudo

- /etc/sudoers文件设置举例:

```
## Allow root to run any commands anywhere
```

```
root ALL=(ALL)      ALL      #找到这一行
```

```
intern01 ALL=(ALL)   ALL      #添加这行
```

#表示 intern01 用户可以在任何地方（远程终端）登录，以任意用户身份执行任何命令

```
## Allow root to run any commands anywhere
```

```
root ALL=(ALL)      ALL      #找到这一行
```

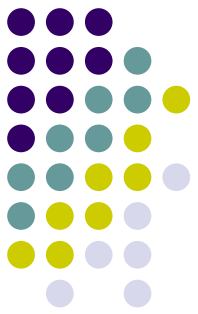
```
intern01 localhost=(root) /sbin/shutdown -h -5
```

```
## Allow root to run any commands anywhere
```

```
root ALL=(ALL)      ALL      #找到这一行
```

```
%group_p 192.168.56.130=(root) NOPASSWD:/sbin/shutdown -h -5
```

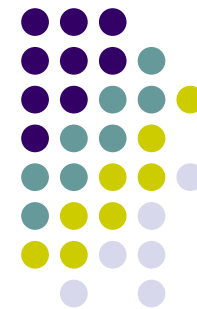
#表示从属于 group_p 用户组的 用户可以在 192.168.56.130 主机上登录，以 root 用户身份执行/sbin/shutdown -5 now 这个命令,并且无需输入用户密码



3.4 用户切换举例

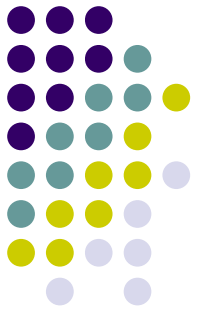
- `su myuser`和`su - myuser`区别, 可以使用`pwd`命令验证
- `su - -c` 用法: `su - -c "tail -n 4 /etc/shadow"`
- `sudo su -`和`su -`区别

本次课程内容

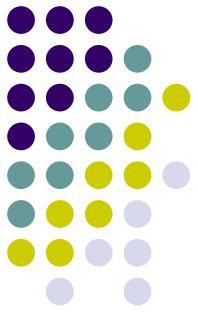


- 进程管理
- 服务与网络管理
- 重定向与管道
- 软件包管理
- vi/vim使用

Part 4 进程管理



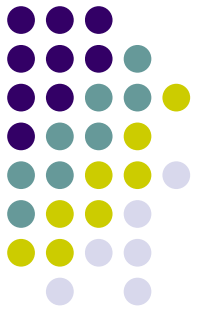
- ps
- top
- pstree
- pgrep
- nice
- jobs
- bg/fg
- kill



4.1 进程分类

● Foreground process 前台进程

- 要求用户启动它们或与它们交互的进程称为前台进程。
- 前台进程不结束，终端就不会出现系统提示符，直到进程终止。
- 缺省情况下，程序和命令作为前台进程运行。



4.1 进程分类

● Background process 后台进程

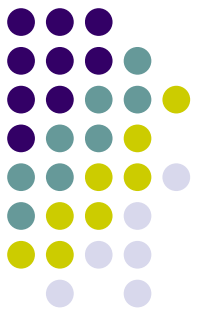
- 独立于用户运行的进程称为后台进程。
- 用户在输入命令行后加上 “&” 字符然后按 <Enter> 键就启动了后台进程。
也是在这里输入输出
- Shell 不等待命令终止，就立即出现系统提示符，让该命令进程在后台运行，用户可以继续执行新的命令。



4.1 进程分类

● Daemon 守护进程

- 总是运行在后台的系统进程。
不再占用终端
- 守护程序通常在系统启动时启动，并且它们一直运行到系统停止。
不会输出屏幕上
- 守护进程常常用于向用户提供各种类型的服务和执行系统管理任务。
- 守护程序进程由 root 用户或 root shell 启动，并且只能由 root 用户停止。

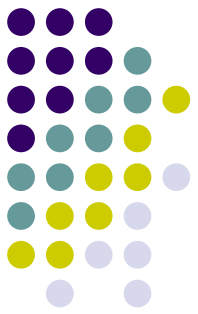


4.2 ps

- ps是“Process Status”的缩写。在命令行输入“ps”后回车就能查看当前系统中正在运行的进程。ps的命令格式如下：

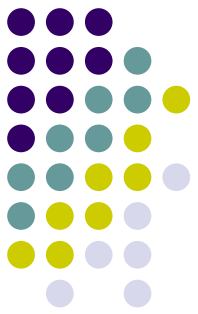
ps [选项] [参数]

选项	说明
a	显示当前终端机下的所有进程，包括其它用户启动的进程
u	以用户的形式，显示系统中的进程
x	忽视终端机，显示所有进程
e	显示每个进程使用的环境变量
r	只列出当前终端机中正在执行的进程



4.3 top

- top的命令格式如下：
top [选项]
- top命令可以实时观察系统的整体运行情况，默认时间间隔为3秒，即每3秒更新一次页面，类似Windows系统中的任务管理器，是一个很实用的系统性能监测工具。



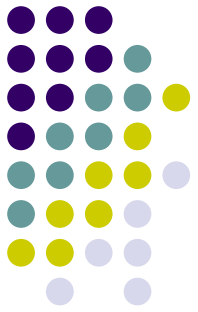
4.4 pstree

- pstree命令以树状图的形式显示系统中的进程，可以直接观察到进程之间的派生关系。pstree命令的格式如下：

pstree [选项]

- pstree命令中的常用选项如表所示。

选项	说明
-a	显示每个进程的完整命令（包括路径、参数等）
-c	不使用精简标识法
-h	列出树状图，特别标明当前正在执行的进程
-u	显示用户名称
-n	使用程序识别码排序（默认以程序名称排序）

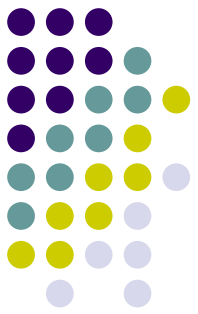


4.5 pgrep

- pgrep命令根据进程名从进程队列中查找进程，查找成功后默认显示进程的pid。pgrep命令的格式如下：

pgrep [选项] [参数]

选项	说明
-o	仅显示同名进程中pid最小的进程
-n	仅显示同名进程中pid最大的进程
-p	指定进程父进程的pid
-t	指定开启进程的终端
-u	指定进程的有效用户id



4.6 nice

- nice命令用于设置Linux系统中进程的nice值，该命令的格式如下：

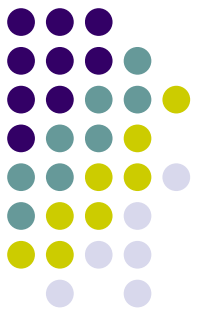
nice [选项] [参数]

- nice选项常用的选项为-n，n表示优先级，是一个整数。

示例：

```
# nice -n 5 bash
```

```
# nice -n 6 top
```



4.7 jobs

- 使用jobs命令可以查看Linux系统中的作业列表及作业状态，进程中的作业也有编号，编号从1开始。Linux系统中作业从用户角度进行编号，进程从系统管理员的角度进行编号。jobs命令的语法格式如下：

jobs [选项] [参数]

- 当选项和参数缺省时，默认显示作业编号、作业状态和启动作业的命令。

选项	说明
-l	显示进程号
-p	仅显示作业对应的进程号
-n	显示作业状态的变化
-r	仅显示运行状态的任务
-s	仅显示停止状态的任务



4.8 bg/fg

- Linux系统中可以使用bg命令和fg命令，使进程在前台和后台之间进行切换。
- bg命令的作用是，将进程放入后台运行，使前台可以执行其它任务。其命令格式如下：

bg 参数

- fg命令的作用与bg相反，是将后台的进程调往前台。其命令格式如下：

fg 作业号

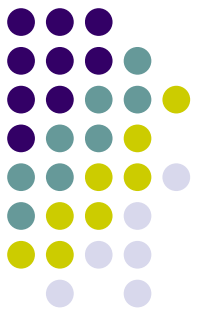


4.9 kill

- kill命令用来终止正在运行的进程，它的工作原理是发送某个信号到指定进程，以终止该进程。kill的命令格式如下：

kill [选项] [参数]

- kill命令的选项一般是“-信号编号”，参数一般是PID。除管理进程外，kill命令也可用于查看系统中的信号。



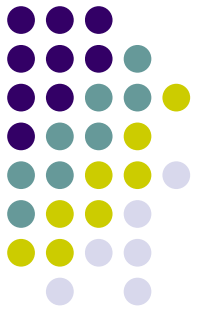
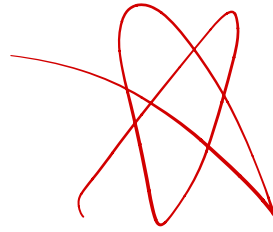
4.10 nohup(no hang up)

- 用于在系统后台不挂断地运行命令，可以在退出账户、关闭终端或是断开远程连接之后，能够继续运行相应的进程。
命令格式如下：

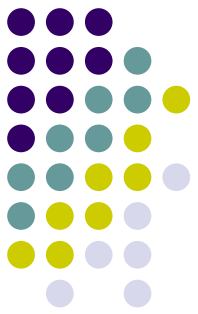
nohup 命令 [&]

- 在默认情况下（非重定向时），会输出一个名叫 nohup.out 的文件到当前目录下
- 要停止运行，可以使用ps命令查找到 nohup 命令的 PID，使用 kill 命令来删除

Part 5 系统管理



- 服务管理
- 软件管理
- 任务管理

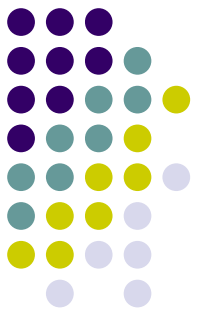


5.1 服务管理

- 服务

服务(service) 本质就是守护进程，是运行在后台，通常都会监听某个端口，等待其它程序的请求，比如(mysql, sshd 防火墙等)，又称为守护进程。

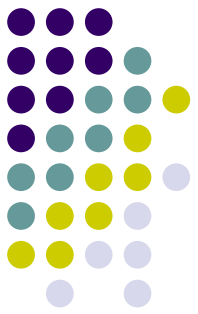
- 系统守护进程，为系统提供某种重要的功能，如 atd、cron、lpd、syslogd、login 等。
- 网络守护进程，向用户提供某种网络服务，如 sshd、httpd、sendmail、xinetd 等。



5.1 服务管理

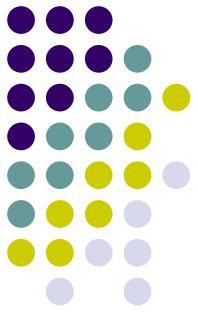
- Linux系统提供了特定的命令，以便管理各种服务，CentOS7及其之后的版本统一使用systemctl命令管理服务。

systemctl 功能 服务名称



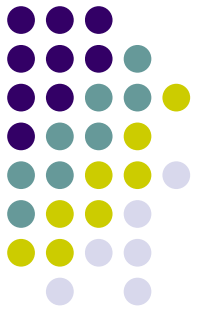
5.1 服务管理

- Linux 系统下共定义了 65536 个可用端口：
- 系统端口（0 - 1023）：众所周知的端口，由IANA（互联网数字分配机构，Internet Assigned Numbers Authority）分配给特定服务；
- 注册端口（1023 - 49151）：用户端口，可用于注册IANA。注册是为了避免端口冲突；
- 动态端口（49152-65535）：动态端口可以分配给服务一段时间，主要由客户端程序使用。



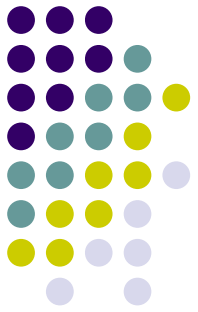
5.1 服务管理

- 常见端口号：
 - FTP - 21
 - SSH - 22
 - TELNET - 23
 - SMTP - 25
 - DNS - 53
 - HTTP - 80



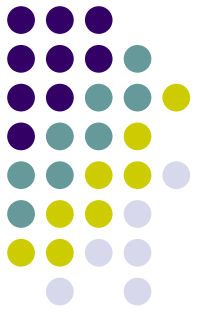
5.1 服务管理

- 常见端口号：
- HTTPS - 443
- SQLServer: 1433
- Oracle: 1521
- MySQL: 3306
- Tomcat: 8080



5.1 服务管理

- systemctl命令功能的基础取值与含义如下：
 - enable: 使指定服务开机自启;
 - disable: 取消指定服务开机自启;
 - start: 启动指定服务;
 - stop: 停止指定服务;
 - status: 检查指定服务运行情况, 列出该服务的详细信息;
 - restart: 重启指定服务;
 - reload: 重新加载指定服务的配置文件



5.1 服务管理

- 1、关闭http服务
systemctl **stop** httpd
- 2、开启http服务
systemctl **start** httpd
- 3、取消http服务开机自启
systemctl **disable** httpd
- 4、使http服务开机自启
systemctl **enable** httpd
- 5、查看http服务的运行状态
systemctl **status** httpd

5.3 任务管理

5.3.2 crontab命令

crontab 文件

5个固定的字段

minute hour day-of-month month-of-year day-of-week commands

最好用绝对路径

参数	描述
minute	分钟 (0 - 59)
hour	小时 (0 - 23)
day-of-month	一个月的第几天 (1 - 31)
month-of-year	一年的第几个月 (1 - 12)
day-of-week	一周的星期几 (0 - 7), 0/7代表周日
commands	需要执行的命令

208

刚刚

不懂

收藏

5.3 任务管理

5.3.2 crontab命令

crontab 文件

在 crontab 文件中输入需要执行的命令和时间

该文件中每行都包括6个域，其中前5个域是指定命令被执行的时间，最后一个域是要被执行的命令

每个域之间使用空格或者制表符分隔，格式如下：

5.3 任务管理

5.3.2 crontab命令

crontab 命令举例

每1分钟执行一次command: `* * * * * command`
 每小时的第3和第15分钟执行: `3,15 * * * * command`
 在上午8点到11点的第3和第15分钟执行: `3,15 8-11 * * * command`
 每隔两天的上午8点到11点的第3和第15分钟执行: `3,15 8-11 */2 * * command`
 每个星期一的上午8点到11点的第3和第15分钟执行: `3,15 8-11 * * 1 command`
 每晚的21:30重启smb: `30 21 * * * /etc/init.d/smb restart`

分钟的字段
- 10秒
不在使用，
连表的用 -
绝对路径
绝对

刚刚

不懂

收藏

5.3 任务管理

5.3.2 crontab命令

crontab 文件

minute hour day-of-month month-of-year day-of-week commands

参数	描述
minute	分钟 (0 - 59)
hour	小时 (0 - 23)

5.3 任务管理

5.3.2 crontab命令

crontab 命令举例

晚上11点到早上7点每隔一小时重启smb: `* 23-7/1 * * *`

`/etc/init.d/smb restart`

每月的4号与每周一到周三的11点重启smb: `0 11 4 * mon-wed`

`/etc/init.d/smb restart`

一月一号的4点重启smb: `0 4 1 jan * /etc/init.d/smb restart`

每小时执行/etc/cron.hourly目录内的脚本: `01 * * * * root run-parts`

`/etc/cron.hourly`

211

刚刚

不懂

收藏

5.3 任务管理

5.3.2 crontab命令

crontab 命令举例

每月1、10、22日的4:45重启smb: `45 4 1,10,22 * * /etc/init.d/smb restart`

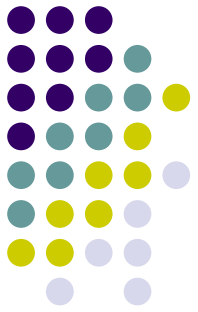
每周六、周日的1:10重启smb: `10 1 * * 6,0 /etc/init.d/smb restart`

每天18:00至23:00之间每隔30分钟重启smb: `0,30 18-23 * * *`

`/etc/init.d/smb restart`

分钟

* 与 小时区别



5.4 网络配置文件

- 网络接口配置文件
- 域名解析配置文件
- 域名和IP映射配置文件
- 主机名配置文件



5.4.1 网络接口配置文件

- 网络接口配置文件——Centos
/etc/sysconfig/network-scripts/ifcfg-ens33

```
TYPE=Ethernet
PROXY_METHOD=none
BROWSER_ONLY=no
BOOTPROTO=dhcp
DEFROUTE=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV6_FAILURE_FATAL=no
IPV6_ADDR_GEN_MODE=stable-privacy
NAME=ens33
UUID=0306fd88-9eb0-4545-98b4-669940963c95
DEVICE=ens33
ONBOOT=yes
```



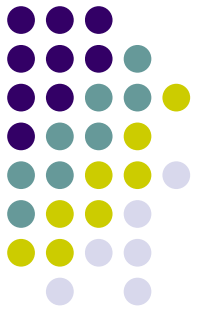
5.4.1 ifcfg-ens33 文件

- 网络接口配置文件——Ubuntu
/etc/netplan/01-network-manager-all.yaml

```
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
```

配置实例

```
network:
  ethernets:
    eno1:
      addresses:
        - 10.12.1.6/21
      gateway4: 10.12.1.254
      nameservers:
        addresses:
          - 114.114.114.114
        search: []
  version: 2
```

5.4.2 DNS配置文件

- DNS 客户端配置文件 ——Centos

/etc/resolv.conf文件

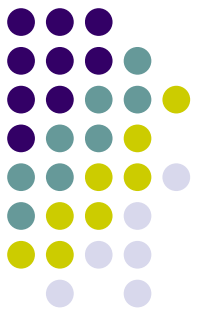
```
# Generated by NetworkManager
search localdomain
nameserver 192.168.179.2
```



5.4.2 DNS配置文件

- DNS 客户端配置文件 ——Ubuntu
/etc/systemd/resolved.conf文件

```
[Resolve]
#DNS=
#FallbackDNS=
#Domains=
#LLMNR=no
#MulticastDNS=no
#DNSSEC=no
#DNSOverTLS=no
#Cache=no-negative
#DNSStubListener=yes
```



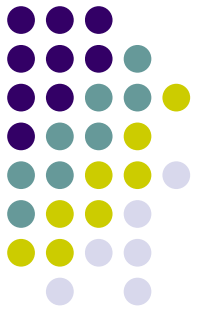
5.4.3 主机名和IP映射文件

- 主机名和IP映射关系文件

/etc/hosts

```
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6

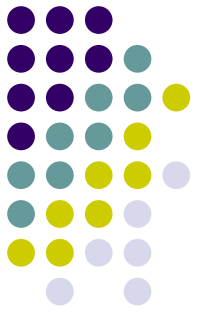
192.168.179.156 node01
```



5.4.4 主机名配置文件

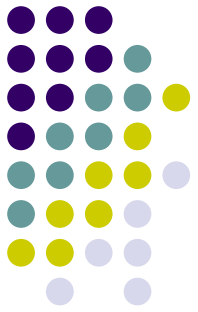
- 主机名配置文件
/etc/hostname

```
localhost.localdomain  
node01
```



5.5 网络工具

- 查看网络端口 `netstat`
- 命令行下载工具 `wget`
- 远程登录 `ssh`
- 文件传输命令 `scp`



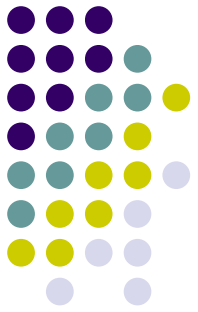
5.5.1 netstat

- 查询网络服务和端口
- 使用格式：
netstat 选项

选项	说明
-a	显示所有Socket链接
-l	显示监听的服务器的Socket
-n	显示IP地址，不显示主机名
-t	仅显示tcp相关连接
-u	仅显示udp相关连接
-p	显示正在使用Socket的程序名

示例：

- `netstat -nulp`



5.5.2 wget

- 用来从指定的URL下载文件
- 使用格式:

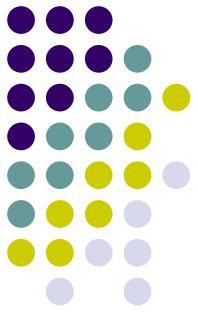
wget 选项 [URL]

选项	说明
-c	断点续传下载文件
-S	打印服务器响应
-O	指定保存的文件名
-P	保存到指定目录

示例:

- #Nginx.org官网下载Nginx-1.13.0.tar.gz包

wget http://nginx.org/download/nginx-1.18.0.tar.gz



5.5.3 ssh

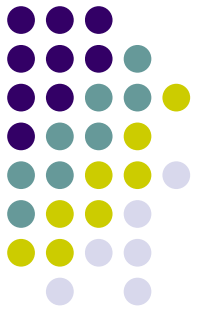
secure shell

- 使用SSH登录
- 使用格式:

ssh 登录主机的用户名@登录主机ip地址

示例:

ssh root@node01



5.5.3 ssh

- 登录方式

- 密码登录
- 密钥登录（免密登录）

利用密钥生成器制作一对密钥（公钥和私钥）。将公钥添加到服务器的某个账户上，然后在客户端利用私钥即可完成认证并登录。

[完整文档](#)



5.5.4 scp

- 使用SSH协议，将文件从一个Linux系统复制到另一个Linux系统
- 使用格式：

scp 远程用户名@ip地址: 文件的绝对路径 本地路径

scp 本地路径 远程用户名@ip地址: 文件的绝对路径

示例：

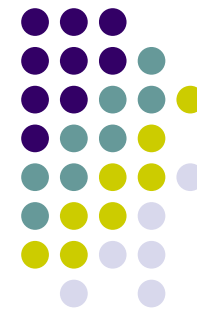
主机之间复制文件

scp /home/mylinux/file root@node01:/tmp

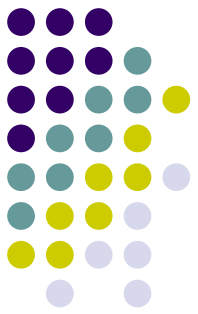
scp -r /etc root@node01:/opt

将整个目录

Part 6 重定向与管道

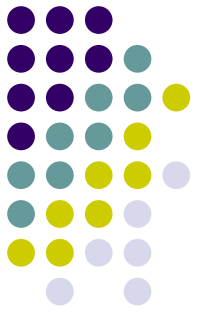


- 标准输入输出流
- 重定向
- 管道



6.1 标准输入输出流

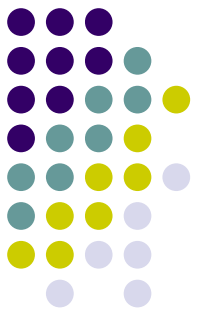
- 标准输入流 (stdin) , 向命令提供输入, 文件描述符是 0。
- 标准输出流 (stdout) , 显示来自命令的输出, 文件描述符是 1。
- 标准错误输出流 (stderr) , 显示来自命令的错误输出, 文件描述符是 2。



6.1 标准输入输出流

- Linux命令执行过程





6.2 输入输出重定向

- 不使用系统默认的标准输入输出流，重新指定命令的输入输出。使用格式：**fd 操作符 文件**

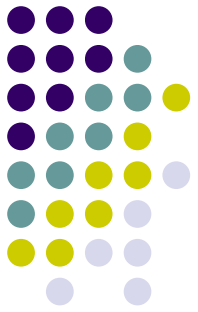
(1) 输入重定向

改变命令的输入流，输入不再来自键盘

操作符是<和<<

<文件名：指定输入来自的文件

<< 分隔符：指定输入来自一对分隔符之间的内容



6.2 输入输出重定向

(1) 输入重定向

- 使用举例：

```
wc < /etc/passwd
```

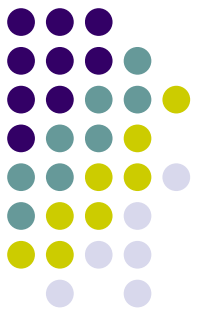
```
wc << END
```

~ 分隔

Hello

World

END



6.2 输入输出重定向

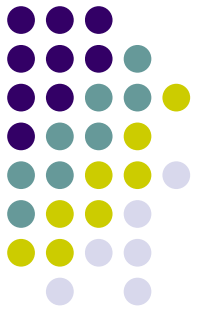
(2) 输出重定向

输出重定向将命令的结果不在屏幕输出，重定向到指定的文件

操作符是 `>` 和 `>>`

`> 文件名`：指定输出到的文件，覆盖写

`>> 文件名`：指定输出到的文件，追加写



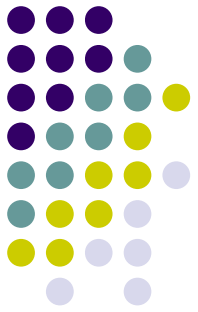
6.2 输入输出重定向

(2) 输出重定向

- 使用举例

`ls -l 1>out.txt`或 `ls -l >out.txt`

`ls -l 1>>out.txt`或 `ls -l >>out.txt`



6.2 输入输出重定向

(3) 错误重定向

- 标准输出和标准错误重定向到一个文件，使用&>和&>>
- 忽略标准输出和标准错误，重定向到/dev/null 文件
- 使用举例：

ls -l 2>err.txt

ls -l 2>>err.txt

ls -l file1 file2 >out 2>&1

ls -l file1 file2 &>file

ls -l file1 file2 &>/dev/null

2 1

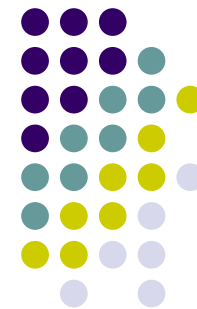
→ 标准输出流

→ 用 & 表示 文件名.

1, 2 都重定向

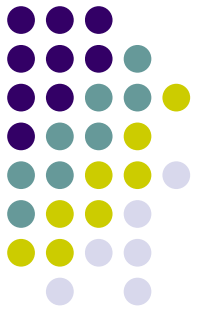
6.2 输入输出重定向

从左向右解析



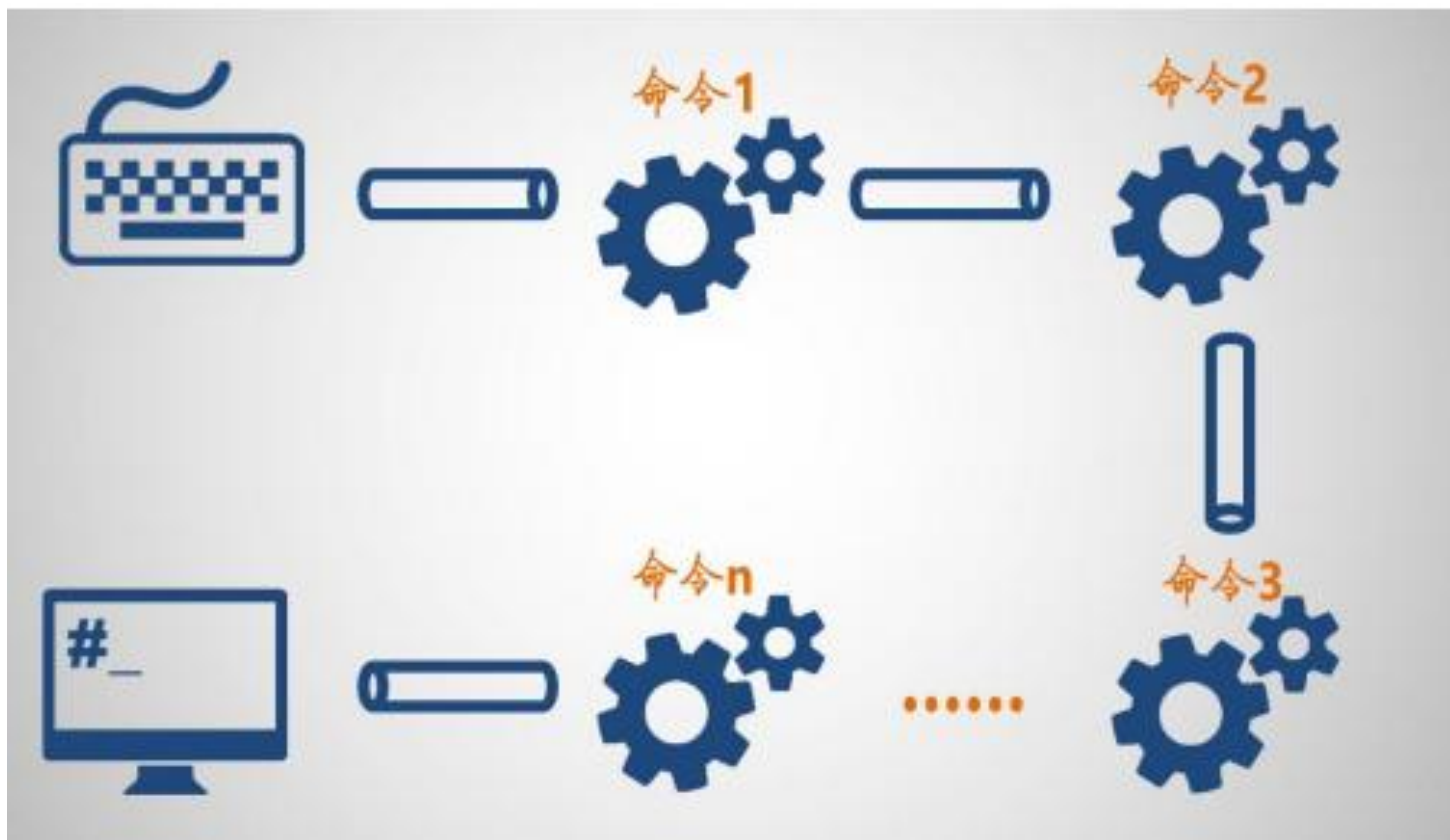
- **Question:** `ls > file 2>&1` 和 `ls 2>&1 >file` 有什么区别?

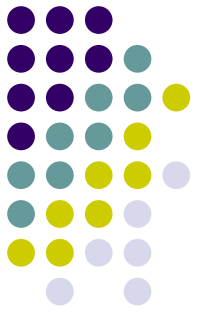
从左向右解析



6.3 管道

- 一种特殊的重定向
- 管道操作符：|





6.3 管道

- 使用要求:

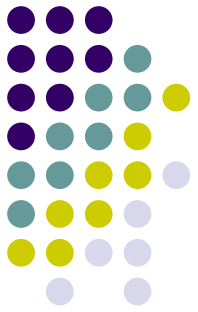
左边命令
只处理前一个命令正确输出，不处理错误输出

管道命令右边命令，必须能够接收标准输入流命令

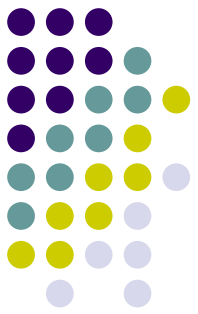
WC

*less
more
!*

Part 7 Vi编辑器

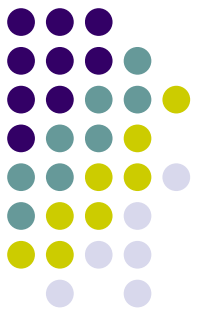


- Vi编辑器介绍
- Vi编辑器使用



7.1 Vi 编辑器介绍

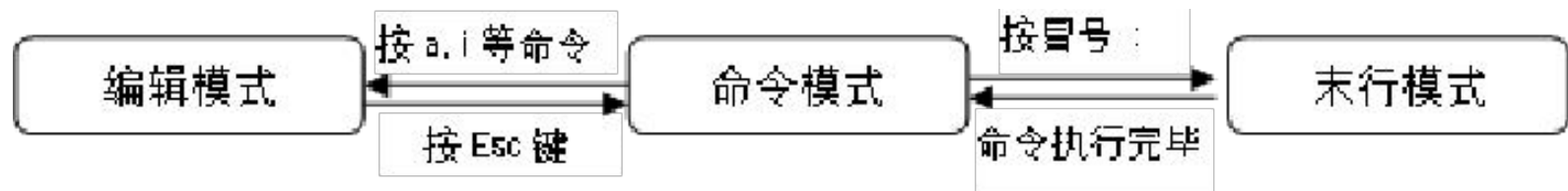
- vi是在UNIX 上被广泛使用的中英文编辑软件，vi是visual editor的缩写，是UNIX提供给用户的一个窗口化编辑环境。
- 进入vi，直接执行vi编辑程序即可。
- 例：\$vi test.c

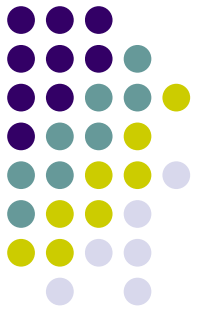


7.2 Vi编辑器使用

1.vi的工作模式

- 输入模式：输入字符作为文本内容
- 命令模式：输入字符为命令，可进行删除、修改、存盘等操作
- 末行模式：命令模式下输入 “: /? ” 三个中任意一个，可移到屏幕最底一行。



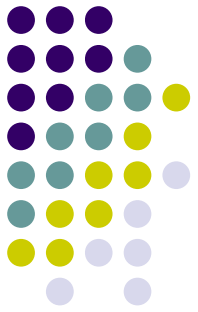


7.2 Vi编辑器使用

(1) 命令模式

- 输入模式下，按ESC可切换到命令模式，常用命令：

: q!	离开vi，并放弃刚在缓冲区内编辑的内容	不保存退出
: wq	将缓冲区内资料写入磁盘中，并离开vi	
: ZZ	同wq	
: x	同wq	
: w	将缓冲区内资料写入磁盘中，但并不离开vi	
: q	离开vi，若文件被修改过，则要被要求确认是否放弃修改的内容，此指令可与：w配合使用	

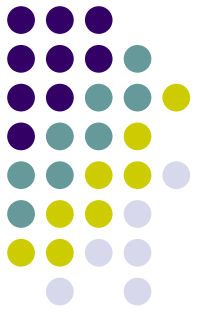


7.2 Vi编辑器使用

(2) 输入模式

- 输入以下命令即可进入vi输入模式

a(append)	在光标之后加入资料
A	在该行之末加入资料
i(insert)	在光标之前加入资料
I	在该行之首加入资料
o(open)	新增一行于该行之下，供输入资料用
O	新增一行于该行之上，供输入资料用
dd	删除当前光标所在行
x	删除当前光标字符
X	删除当前光标之前字符
U	撤消
F	查找
ESC	离开输入模式

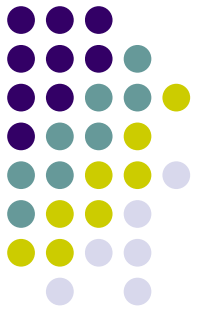


7.2 Vi编辑器使用

2.Vi其他功能命令

(1) 复制粘贴

yw	将光标所在之处到字尾的字符复制到缓冲区
yy	复制光标所在行到缓冲区
#yy	如：6yy表示拷贝从光标所在行往下数6行文字
p	将缓冲区内的字符贴到光标所在位置

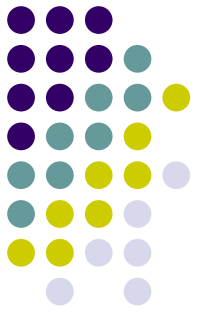


7.2 Vi编辑器使用

2.Vi其他功能命令

- (2) 查找/替换

?字符串	从当前光标位置开始向后查找字符串
/字符串	从当前光标位置开始向前查找字符串
n	继续上一次查找
Shift+n	以相反的方向继续上一次查找



7.2 Vi编辑器使用

2.Vi其他功能命令

- (3) 环境设置 /etc/vimrc文件，修改家目录文件.vimrc文件

:set ai	自动缩进，每一行开头都与上一行的开头对齐
:set nu	编辑时显示行号
:set dir=./	将交换文件.swp保存在当前目录
:set sw=4	设置缩进的字符数为4
:syntax on 或者 :syntax=on	开启语法着色